Market Labs

Dokumentasi Teknis-Fungsional Per Menu

Build: 11 October 2026 09:15 · Versi dokumen: 1.0

Sumber: docs/*.md · idx-web & idx-scraper

Daftar Isi

Market Labs — Dokumentasi Menu **5**

Arsitektur Ringkas 5

Indeks Menu 5

Diagram Alur Data (Menu Kompleks) 6

Mapping Menu → Endpoint → Modul Backend → Tabel → Job 7

Penjadwalan (APScheduler, WIB) 8

Cara Menjalankan Backend (agar semua data update) 9

Delete-Zone (Fitur UI yang Sengaja Dihapus) 9

Konvensi 9

Daftar File 9

Build PDF 10

Aktivitas Broker (/broker-activity) **11**

1. Ringkasan Fungsi 11

2. Route & Berkas 11

3. Penjelasan Per-Card / Per-Panel 11

4. Data Flow (end-to-end) 12

5. Cara Pengambilan / Update Data 12

6. Dependensi & Relasi Menu 12

7. Catatan Batasan & Edge Case 12

Aliran Dana Emiten (/flow + /flow/[code]) **13**

1. Ringkasan Fungsi 13

2. Route & Berkas 13

3. Penjelasan Per-Card / Per-Panel 13

4. Data Flow (end-to-end) 14

5. Cara Pengambilan / Update Data 14

6. Dependensi & Relasi Menu 14

7. Catatan Batasan & Edge Case 14

Backtest (/backtest) **15**

1. Ringkasan Fungsi 15

2. Route & Berkas 15

3. Penjelasan Per-Card / Per-Panel 15

4. Data Flow (end-to-end) 16

5. Cara Pengambilan / Update Data 17

6. Dependensi & Relasi Menu 17

7. Catatan Batasan & Edge Case 17

Dashboard (/) **18**

1. Ringkasan Fungsi 18

2. Route & Berkas 18

3. Penjelasan Per-Card 18

4. Data Flow (end-to-end) 19

5. Cara Pengambilan / Update Data 20

6. Dependensi & Relasi Menu 20

7. Batasan & Edge Case 20

Faktor & Kalibrasi (/faktor) **21**

1. Ringkasan Fungsi 21

2. Route & Berkas 21

3. Penjelasan Per-Card / Per-Panel 21

4. Data Flow (end-to-end) 22

5. Cara Pengambilan / Update Data 22

6. Dependensi & Relasi Menu 22

7. Catatan Batasan & Edge Case 22
Foreign Flow (/foreign) **23**
1. Ringkasan Fungsi 23
2. Route & Berkas 23
3. Penjelasan Per-Card / Per-Panel 23
4. Data Flow (end-to-end) 23
5. Cara Pengambilan / Update Data 23
6. Dependensi & Relasi Menu 23
7. Catatan Batasan & Edge Case 23
Framework Analisis Saham Multi-Agen **24**
1. Tim Analis 24
2. Tim Peneliti (Researcher Team) 24
3. Agen Trader 24
4. Manajemen Risiko dan Manajer Portofolio 24
5. Tim Engineering 24
6. Tim Frontend 24
7. Tim Trainer 24
Catatan Status 25
Halaman Detail Emiten (/stock/[code]) **26**
1. Ringkasan Fungsi 26
2. Route & Berkas 26
3. Penjelasan Per-Card / Per-Panel 26
4. Data Flow (end-to-end) 27
5. Cara Pengambilan / Update Data 27
6. Dependensi & Relasi Menu 28
7. Catatan Batasan & Edge Case 28
Jejak Sinyal (/jejak-sinyal) **29**
1. Ringkasan Fungsi 29
2. Route & Berkas 29
3. Penjelasan Per-Card 29
4. Data Flow (end-to-end) 30
5. Cara Pengambilan / Update Data 30
6. Dependensi & Relasi Menu 30
7. Catatan Batasan & Edge Case 30
Pantau (/pantau) **31**
1. Ringkasan Fungsi 31
2. Route & Berkas 31
3. Penjelasan Per-Card 31
4. Data Flow (end-to-end) 32
5. Cara Pengambilan / Update Data 32
6. Dependensi & Relasi Menu 32
7. Catatan Batasan & Edge Case 32
Radar Smart Money (/radar + /radar/[code]) **33**
1. Ringkasan Fungsi 33
2. Route & Berkas 33
3. Penjelasan Per-Card / Per-Panel 33
4. Data Flow (end-to-end) 34
5. Cara Pengambilan / Update Data 34
6. Dependensi & Relasi Menu 34
7. Catatan Batasan & Edge Case 34
Rekomendasi Beli (/rekomendasi) **36**
1. Ringkasan Fungsi 36
2. Route & Berkas 36

3. Penjelasan Per-Card 36
4. Data Flow (end-to-end) 36
5. Cara Pengambilan / Update Data 36
6. Dependensi & Relasi Menu 37
7. Catatan Batasan & Edge Case 37
Ruang Keputusan (/keputusan, /keputusan/[code]) 38
1. Ringkasan Fungsi 38
2. Route & Berkas 38
3. Penjelasan Per-Card 38
4. Data Flow (end-to-end) 39
5. Cara Pengambilan / Update Data 39
6. Dependensi & Relasi Menu 39
7. Catatan Batasan & Edge Case 39
Screener (/screener) 40
1. Ringkasan Fungsi 40
2. Route & Berkas 40
3. Penjelasan Per-Card 40
4. Data Flow (end-to-end) 40
5. Cara Pengambilan / Update Data 40
6. Dependensi & Relasi Menu 41
7. Catatan Batasan & Edge Case 41
Sektor (/sectors) 42
1. Ringkasan Fungsi 42
2. Route & Berkas 42
3. Penjelasan Per-Card / Per-Panel 42
4. Data Flow (end-to-end) 42
5. Cara Pengambilan / Update Data 42
6. Dependensi & Relasi Menu 43
7. Catatan Batasan & Edge Case 43
Sentimen (/sentimen) 44
1. Ringkasan Fungsi 44
2. Route & Berkas 44
3. Penjelasan Per-Card / Per-Panel 44
4. Data Flow (end-to-end) 44
5. Cara Pengambilan / Update Data 44
6. Dependensi & Relasi Menu 45
7. Catatan Batasan & Edge Case 45
Sumber Data Berita Market 46
1. Pasar Indonesia (IHSG, saham, obligasi, makro BI) 46
2. Global Macro & Institutional 46
3. Crypto (BTC/ETH/altcoin + sentimen) 46
4. Gold / Komoditas 46
5. Sentimen Sosial (untuk "whale mode" sentiment read) 46
Catatan 46

Market Labs — Dokumentasi Menu

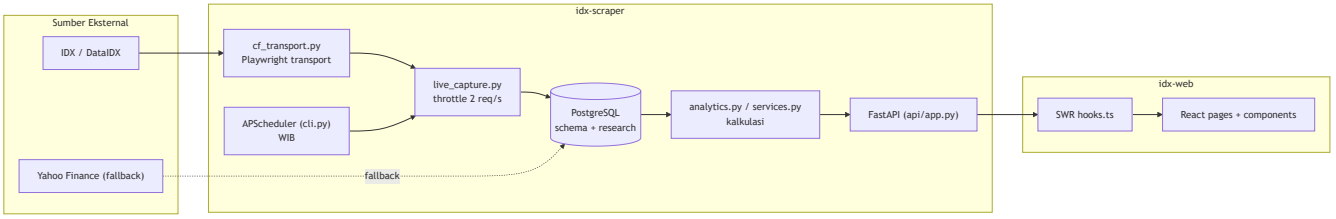
Dokumentasi teknis-fungsional per menu untuk aplikasi **Market Labs** (platform data pasar saham IDX). Sumber kebenaran daftar menu adalah `components/Sidebar.tsx` (GROUPS, baris 33–70).

Dokumen ini dipakai oleh tiga audiens:

- **Developer baru** — onboarding: peta route, endpoint, tabel, dan job.
- **Analis / PM** — memahami fungsi tiap card dan dari mana angkanya.
- **Maintainer** — tahu kapan data dibuat, job mana yang mengisi, dan bagaimana memperbaruinya.

Arsitektur Ringkas

- `idx-web` — Frontend Next.js (App Router) + Recharts + Tailwind. **Presentation-only**: nol kalkulasi bisnis. Semua data di-fetch sebagai JSON lewat SWR (`lib/hooks.ts`, interval default 30 detik). Tipe mirror di `lib/types.ts`, fetch client di `lib/api.ts`.
- `idx-scraper` — Backend Python/FastAPI + PostgreSQL + APScheduler (timezone WIB). Sumber kalkulasi & penyimpanan. API di `src/idx_scraper/api/`, scheduler di `cli.py`.
- Base URL backend: `NEXT_PUBLIC_API_BASE_URL` (default `http://localhost:8000`).
- **Chrome global** (dipasang di semua halaman): sidebar bisa diciutkan menjadi rail, tema terang/gelap, badge kesegaran data, dan command palette (Ctrl/Cmd+K). Rincian di bagian Chrome Global.



Indeks Menu

Group	Route	Menu	Fungsi (1 baris)	Card	Sumber data utama	Update
Utama	`/`	Dashboard	Ringkasan pasar: IHSG, regime, narasi, leaderboard, sinyal	8	`index_quotes`, `index_summary_daily`, `research.latest_pit`	on-demand + SWR 30s
Utama	`/pantau`	Pantau	Watchlist + alert + chart	5	`watchlist`, `stock_quotes`	on-demand + SWR 30s
Screening	`/screener`	Screener	Filter saham multi-kriteria	3	`research.latest_pit`	on-demand + SWR 30s
Analisis	`/rekomendasi`	Rekomendasi Beli	Papan kandidat + track record	3	`research.recommendation_log`	job harian `idx recommend`
Analisis	`/keputusan`	Ruang Keputusan	Verdict per emiten + event study	5	`research.latest_pit`, `signal_log`	on-demand (cache 1h)
Analisis	`/ruang-keputusan`	Ruang Keputusan	Skor keputusan + konteks	2	`research.latest_pit` + `orderbook`	on-demand + SWR 300s
Analisis	`/jejak-sinyal`	Jejak Sinyal	Track record sinyal vs pasar	4	`research.signal_log`	job `idx signal-log`
Analisis	`/backtest`	Backtest	Simulasi point-in-time	8	`research.prices_pit`	on-demand (POST, tanpa tulis DB)
Analisis	`/faktor`	Faktor & Kalibrasi	Bobot IC, regime, registry faktor	6	`research.factor_ic_history`, `regime_daily`	job `idx ic` (harian 06:30)
Flow	`/radar`	Radar Smart Money	Akumulasi/distribusi asing	6	`research.latest_pit`	on-demand + SWR 300s
Flow	`/foreign`	Foreign Flow	Arus asing harian + top	2	`research.latest_pit`	on-demand + SWR 30s
Flow	`/flow`	Aliran Dana Emiten	Timeline aliran per emiten	4	`research.broker_daily`	on-demand + SWR 300s
Flow	`/sentimen`	Sentimen	Skor posisi dari jejak transaksi	3	`research.latest_pit` + `orderbook`	on-demand (cache 1h)
Flow	`/broker-activity`	Aktivitas Broker	Skor aliran + uji IC + struktur broker	4	`research.broker_daily`, `factor_ic_history`	on-demand + SWR 300s
Sektor	`/sectors`	Sektor	Rotasi sektor + peta kekuatan	3	`research.latest_pit`	on-demand + SWR 30s

Halaman Detail & Tambahan

route	Tipe	Isi
`/stock/[code]`	Dinamis	Semua card analitik satu emiten (SmartMoneyBanner, chart, signals, ownership, foreign flow, event study, history)
`/keputusan/[code]`	Dinamis	Workspace keputusan per emiten (verdict, jejak asing, level pembatalan, base rate)

route	Tipe	Isi
`/flow/[code]`	Dinamis	Workspace aliran dana per emiten (ForeignFlowCard, FlowTimelineCard, BrokerFlowCard)
`/radar/[code]`	Dinamis	SmartMoneyBanner + track record + back-link
(semua) `error`	Status	`app/error.tsx`: tampilan galat dengan tombol coba lagi dan `ThemeToggle`
(semua) `loading`	Status	`app/loading.tsx`: skeleton saat route dimuat
(semua) `not-found`	Status	`app/not-found.tsx`: halaman 404 dengan tautan kembali

> Route `/watchlist` tidak terdaftar di `Sidebar.tsx` — watchlist diakses via menu **Pantau**.

Chrome Global

Komponen yang tampil di semua halaman, tidak termasuk dalam tabel menu di atas. Perilaku di bawah ini sudah dikonfirmasi terhadap UI yang berjalan di http://localhost:3000 (2026-10-11).

Fitur	Implementasi	Perilaku	Penyimpanan lokal
Sidebar rail	`components/Sidebar.tsx` (`SIDEBAR_KEY`, baris 73)	Sidebar bisa diciutkan menjadi rail ikon. Grup: Utama, Screening, Analisis, Flow, Sektor	`ml-sidebar-collapsed` (`1` = ciut)
Tema terang/gelap	`components/ThemeToggle.tsx` (`KEY`, baris 8)	Default gelap. Skrip inline di `app/layout.tsx` (`THEME_SCRIPT`) menerapkan kelas sebelum render agar tidak ada kilatan (anti-FOUC)	`ml-theme` (`dark` / `light`)
Badge kesegaran data	`components/GlobalLastUpdated.tsx`, dirender di Topbar lewat `Sidebar.tsx` (baris 172)	Menampilkan waktu data terakhir dari `useMarketOverview()` (`lib/hooks.ts`, baris 59) memakai komponen `LastUpdated`. Teks awal: "Memuat data..."	—
Command palette	`components/CommandPalette.tsx`, dipasang di `app/layout.tsx`	Buka dengan Ctrl/Cmd+K, tutup dengan Esc. Berisi daftar menu (dengan kata kunci) dan pencarian emiten lewat `searchStocks()` → `GET /api/search`	—

Sinkronisasi filter Screener: filter di `app/screener/page.tsx` ditulis ke query string URL (`useSearchParams` + `useRouter`), sehingga tautan hasil filter bisa dibagikan dan dibuka ulang.

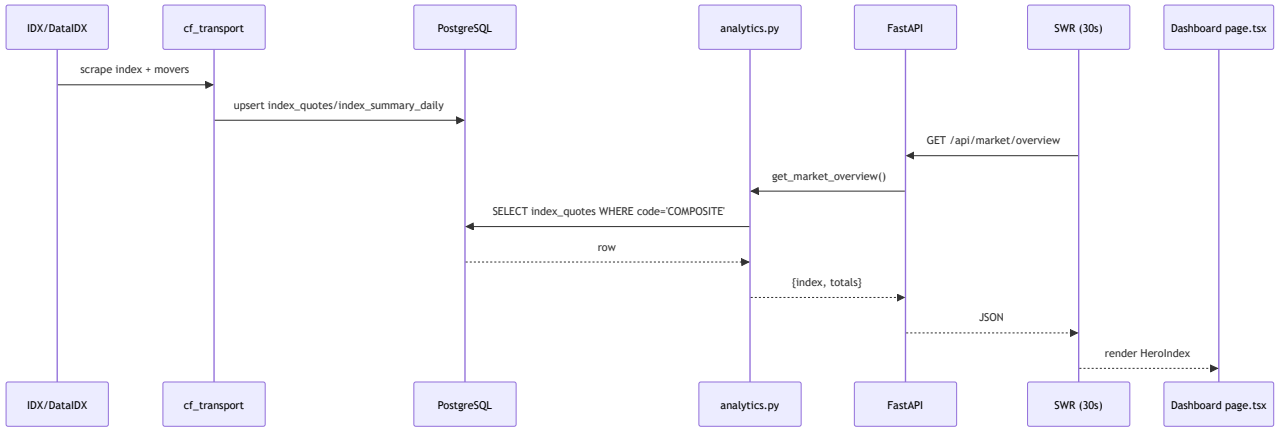
Logo emiten dan broker: `components/TickerLogo.tsx` membaca `manifest.json` dari folder yang ditunjuk prop `basePath`, lalu memakai file lokal di folder tersebut. Bila file tidak ada, ditampilkan monogram kode.

- Emiten: `basePath="/logos"` (default), file `public/logos/{CODE}.png`, manifest `public/logos/manifest.json`. Dataset: 912 logo, 51 monogram (dari 963 kode). Sumber TradingView. Dipakai di Top Leaders, Market Overview, Watchlist, Radar, Screener, Foreign Flow, dan Smart Money.
- Broker: `basePath="/logos/brokers"`, file `public/logos/brokers/{CODE}.jpg`, manifest `public/logos/brokers/manifest.json`. Dataset: 88 logo (dari 88 kode di `research.broker_daily`), tanpa fallback. Sumber `idx.co.id/StaticData/Brokers/Logo/{CODE}.jpg`, diambil lewat `scripts/fetch_broker_logos.py` (butuh Chrome; lihat README idx-scraper). Dipakai di panel Top Broker (`components/TopBrokersPanel.tsx`).

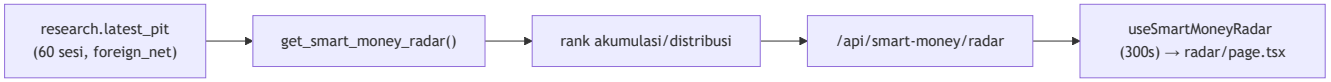
Status hak cipta logo (emiten maupun broker) tetap `[BELUM TERVERIFIKASI]`. Kode broker `PC` disimpan sebagai `.bmp` (sumber mengembalikan BMP di balik URL `.jpg`) dan tetap tampil normal karena UI memakai field `file` dari manifest.

Diagram Alur Data (Menu Kompleks)

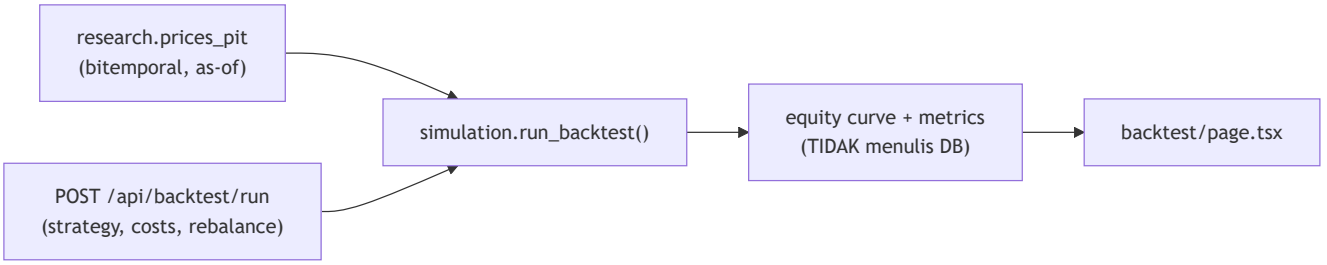
Dashboard



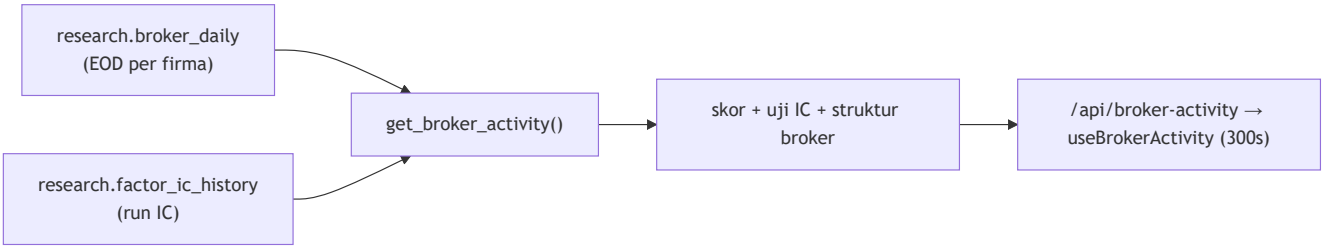
Radar Smart Money



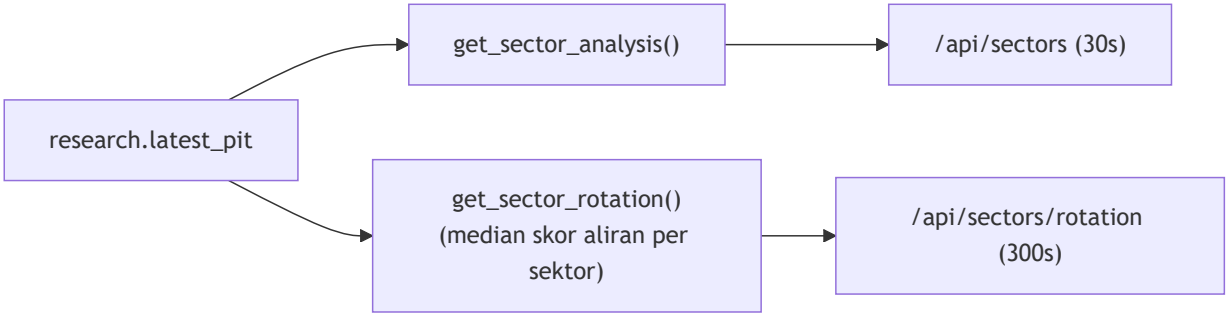
Backtest



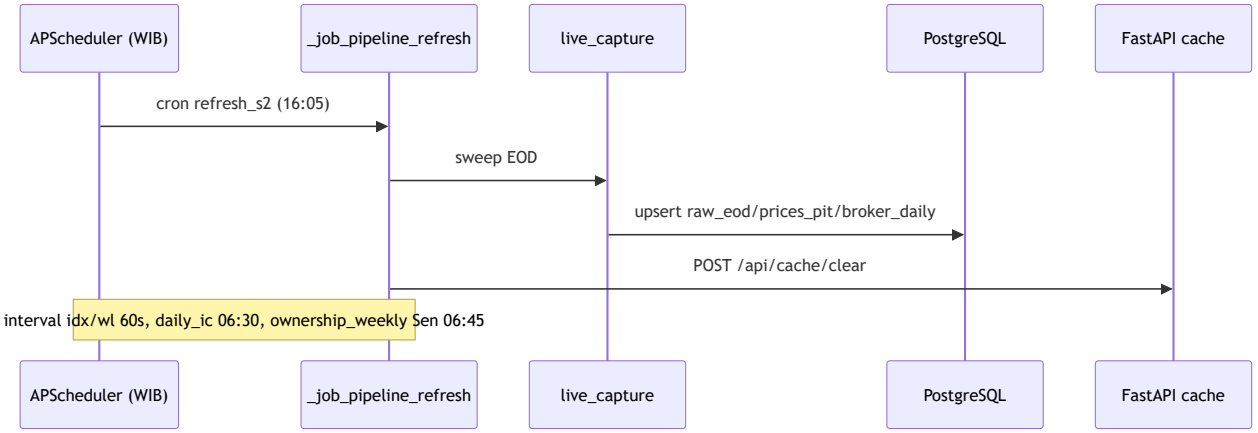
Aktivitas Broker



Sektor



Sekuens Update Data



Mapping Menu → Endpoint → Modul Backend → Tabel → Job

Menu	Endpoint	Modul	Tabel	Job/Penjadwalan
Dashboard	`/api/market/overview`	`services.get_market_overview`	`index_quotes`	interval `idx` 60s
Dashboard	`/api/market/trade-summary`	`services.get_market_trade_summary`	`research.market_segment_daily`	EOD refresh
Dashboard	`/api/market/regime`	`analytics.get_market_regime`	`index_summary_daily`, `research.latest_pit`	on-demand
Dashboard	`/api/market/leaders`	`services.get_market_leaders`	`stock_summary_daily` / `stock_quotes`	interval `wl` 60s
Dashboard	`/api/market/top-brokers`	`services.get_top_brokers`	`research.broker_daily`	EOD refresh
Dashboard	`/api/signals`	`services.get_signals`	`research.latest_pit`	on-demand
Dashboard	`/api/stocks/{code}/technical`	`services.get_technical_chart`	`stock_daily`, `stock_quotes`	interval `wl`
Dashboard	`/api/market/narration`	`analytics.get_market_narration`	`research.latest_pit`	on-demand
Pantau	`/api/watchlist` (GET/POST/PUT/DELETE)	`services.get_watchlist`	`watchlist`	interval `wl` 60s
Pantau	`/api/alerts` (+ test)	`alerts`	alert store	interval `alerts` 300s (bila Telegram aktif)
Screener	`/api/screener`	`analytics.get_screener`	`research.latest_pit`	on-demand
Rekomendasi	`/api/recommendations`	`analytics.get_recommendations`	`research.recommendation_log`	job harian `idx recommend`
Rekomendasi	`/api/recommendations/track`	`analytics.get_recommendation_track`	`research.recommendation_log`	job harian
Rekomendasi	`/api/stocks/{code}/recommendation`	`analytics.get_stock_recommendation`	`research.recommendation_log`	on-demand
Ruang Keputusan	`/api/stocks/{code}/decision`	`analytics.get_stock_decision`	`research.latest_pit`, `signal_log`	on-demand (cache 1h)
Valuasi (endpoint backend, tanpa halaman UI)	`/api/valuation`	`analytics.get_valuation`	`research.latest_pit`	on-demand
Jejak Sinyal	`/api/signals/track`	`analytics.get_signal_track`	`research.signal_log`	job `idx signal-log`
Backtest	`/api/backtest/config + /run`	`simulation`	`research.prices_pit`	on-demand (POST)
Faktor	`/api/factors/overview`	`analytics.get_factors_overview`	`research.factor_ic_history`	job harian `daily_ic` 06:30
Faktor	`/api/market/regime/history`	`analytics.get_regime_history`	`research.regime_daily`	idempoten, backfill dari `index_summary_daily`
Radar	`/api/smart-money/radar`	`analytics.get_smart_money_radar`	`research.latest_pit`	on-demand
Radar	`/api/smart-money/track-record`	`analytics.get_smart_money_track_record`	`research.latest_pit`	on-demand
Radar	`/api/smart-money/patterns`	`analytics.get_smart_money_patterns_board`	`research.latest_pit`	on-demand
Radar	`/api/smart-money/verdicts`	`analytics.get_smart_money_verdicts`	`research.latest_pit`	on-demand
Radar	`/api/stocks/{code}/smart-money`	`analytics.get_stock_smart_money`	`research.latest_pit`	on-demand
Foreign Flow	`/api/foreign-flow`	`analytics.get_foreign_flow`	`research.latest_pit`	on-demand
Foreign Flow	`/api/stocks/{code}/foreign-flow`	`analytics.get_stock_foreign_flow`	`research.latest_pit`	on-demand
Aliran Dana	`/api/broker-flow`	`analytics.get_broker_flow`	`research.broker_daily`	EOD refresh
Aliran Dana	`/api/stocks/{code}/broker-activity`	`analytics.get_stock_broker_activity`	`research.broker_activity`	EOD refresh
Sentimen	`/api/sentiment`	`analytics.get_sentiment`	`research.latest_pit`, `orderbook`	on-demand
Aktivitas Broker	`/api/broker-activity`	`analytics.get_broker_activity`	`research.broker_daily`	EOD refresh
Aktivitas Broker	`/api/stocks/{code}/brokers`	`services.get_broker_concentration`	`research.broker_daily`	EOD refresh
Sektor	`/api/sectors`	`analytics.get_sector_analysis`	`research.latest_pit`	on-demand
Sektor	`/api/sectors/rotation`	`analytics.get_sector_rotation`	`research.latest_pit`	on-demand
(semua)	`/api/cache/clear`	`analytics.clear_research_cache`	—	dipanggil job refresh
Command Palette (semua)	`/api/search?q=&limit=`	`analytics.search_stocks` (analytics.py baris 1626)	`research.latest_pit`	on-demand, saat mengetik di palette
(semua)	`/health`	—	—	—

Penjadwalan (APScheduler, WIB)

Sumber: `cli.py` `cmd_serve`. Semua job refresh memanggil `POST /api/cache/clear`.

Job ID	Jadwal	Tugas
`idx`	interval, `IDX_INDEX_INTERVAL` (default 60s)	refresh index
`wl`	interval, `IDX_WATCHLIST_INTERVAL` (default 60s)	refresh watchlist
`alerts`	interval, `IDX_ALERT_INTERVAL` (default 300s)	evaluasi aturan alert (bila Telegram aktif)
`refresh_preopen`	Mon–Fri 08:45	pra-buka (`do_eod=False`)
`refresh_s1`	Mon–Thu 12:00	sesi 1 (EOD)
`refresh_s1_fri`	Fri 11:30	sesi 1 Jumat (EOD)
`refresh_s2`	Mon–Fri 16:05	sesi 2 / EOD
`refresh_s2_late`	Mon–Fri 16:20	EOD susulan
`health_eod`	Mon–Fri 17:00	cek kesehatan EOD
`refresh_weekend_am`	Sat–Sun 08:45	refresh akhir pekan
`refresh_weekend_pm`	Sat–Sun 16:00	refresh akhir pekan
`daily_ic`	Mon–Fri 06:30	hitung IC faktor → `factor_ic_history`
`ownership_weekly`	interval 1 minggu, mulai Sen 06:45	snapshot ownership

> **Startup catchup**: saat serve mulai, backend mengecek `prices_pit` dan `index_quotes`; bila kosong/stale ia memicu refresh (lihat `cli.py` sekitar baris 1180).

Cara Menjalankan Backend (agar semua data update)

1. **Aktifkan Postgres.** Set `IDX_STORAGE=postgres` di `.env` (jangan commit isi `.env`).
2. **Jalankan serve** (launcher + child + scheduler):

   ```
   cd d:\Project\market-labs\idx-scraper
   .\scripts\start-serve.bat
   ```

   Jangan jalankan proses `score_eval` lain secara paralel tanpa alasan.
3. **Cek health**: `curl http://localhost:8000/health` → 200.
4. **Pastikan transport Chrome** untuk data live: profile `IDX_CHROME_PROFILE_DIR` (mis. `p3`). Bila Chrome lain memegang profile → cf-transport gagal ("existing browser session"). Cooldown revive 600s membatasi pengulangan.
5. **Biarkan scheduler berjalan minimal satu siklus EOD** (`refresh_s2` 16:05 WIB) agar `raw_eod/prices_pit/broker_daily` terisi.
6. **Backfill manual** bila perlu: `idx eod <YYYYMMDD>`, `idx ic`, `idx signal-log`, `idx recommend`.

Delete-Zone (Fitur UI yang Sengaja Dihapus)

Jangan dibuat ulang tanpa keputusan baru. Backend terkait tetap dipakai oleh fitur lain.

Path / konsep	Dihapus pada	Alasan	Backend yang tetap	Syarat buat ulang
`app/valuation/`, `docs/valuasi.md`, `useValuation`	2026-10-11 (commit `7a08ca3`)	Fitur UI Valuasi dihapus	`GET /api/valuation` tetap (dipakai screener & Hold Check)	Keputusan baru + ADR
`app/hold-check/`, `components/HoldCheckPanel.tsx`, `docs/hold-check.md`	2026-10-11 (commit `4ae0dcc`)	Fitur UI Hold Check dihapus	`GET /api/hold-check` tetap (dipakai Ruang Keputusan)	Keputusan baru + ADR

Konvensi

- **Versi padat**: satu file per menu dengan struktur 7 bagian baku; detail identik tidak diulang antar menu.
- Rujukan `file:line` untuk endpoint, komponen, dan rumus.
- Item yang belum dapat diverifikasi dari kode ditandai `[BELUM TERVERIFIKASI]`.
- Tidak ada rahasia (kredensial/token/DSN) dalam dokumen; hanya nama variabel env.

Daftar File

- dashboard.md
- pantau.md
- screener.md
- rekomendasi.md
- ruang-keputusan.md
- jejak-sinyal.md
- backtest.md
- faktor.md
- radar-smart-money.md
- foreign-flow.md
- aliran-dana-emiten.md

- sentimen.md
- aktivitas-broker.md
- sektor.md
- halaman-detail.md
- framework-agen.md — peran agen (Trading, Frontend, Trainer)
- sumber-data.md — daftar sumber berita market

Build PDF

```
cd d:\Project\market-labs\idx-web\docs
& "d:\Project\market-labs\idx-scraper\.venv\Scripts\python.exe" build-pdf.py
```

Output: `Market-Labs-Dokumentasi-Menu.pdf`

Aktivitas Broker (`/broker-activity`)

1. Ringkasan Fungsi

Peringkat emiten berdasarkan skor akumulasi broker (kombinasi faktor aliran yang lolos uji IC), disertai struktur konsentrasi broker pasar dan uji statistik faktor. Menjawab: "faktor aliran mana yang terbukti, dan emiten mana yang skornya tertinggi hari ini?" Pengguna: analis dan PM. Skor hanya tampil bila tervalidasi.

2. Route & Berkas

- URL: `/broker-activity` → `app/broker-activity/page.tsx`
- Komponen: `FlowTimelineCard` (components/FlowTimelineCard.tsx), `BrokerFlowCard` (components/BrokerFlowCard.tsx), `Card`
- Hook: `useBrokerActivity(25)` (hooks.ts:151, refresh 300 detik); `useBrokerFlow` (hooks.ts:117)
- Backend: `GET /api/broker-activity?limit=25` (app.py:234) → `get_broker_activity` (analytics.py:2333) → `broker_activity_snapshot` (analytics.py:2192)

3. Penjelasan Per-Card / Per-Panel

Peringkat Skor Akumulasi

Field	Isi
Nama card	Peringkat Skor Akumulasi
Komponen	broker-activity/page.tsx ~baris 355–363
Fungsi	Daftar emiten urut skor 0–100 dari faktor aliran yang tervalidasi
Sumber data	`GET /api/broker-activity` → `get_broker_activity` (analytics.py:2333); skor dari `broker_activity_snapshot` (satu sumber bersama Screener dan Ruang Keputusan)
Rumus/Logika	Composite faktor yang lolos gate: |mean IC| ≥ 0,05 dan |ICIR| ≥ 0,5; arah mengikuti tanda IC. Skor relatif terhadap pasar hari itu. Pilih emiten untuk timeline (klik baris)
Periode/Window	Lookback panel 60 hari; horizon bobot = `WEIGHT_HORIZON`
Interaksi	Klik baris → timeline aliran dana muncul di bawah
Empty/Loading/Error	Belum tervalidasi → `validated=false`, `rows` kosong, status "belum tervalidasi"

Uji Statistik Faktor (IC)

Field	Isi
Nama card	Uji Statistik Faktor (IC)
Komponen	broker-activity/page.tsx ~baris 364–373
Fungsi	Faktor mana yang layak dipakai; faktor gagal tetap ditampilkan tapi tanpa bobot
Sumber data	`broker_activity_snapshot.factors` ← `research.factor_ic_history` (run terbaru)
Rumus/Logika	`ba.select_factors` (research/broker_activity.py) menerapkan gate IC
Periode/Window	Run IC terbaru; horizon `WEIGHT_HORIZON`
Interaksi	—
Empty/Loading/Error	Pesan "Belum ada run IC" bila tabel kosong

Struktur Broker Pasar

Field	Isi
Nama card	Struktur Broker Pasar
Komponen	broker-activity/page.tsx ~baris 374–380
Fungsi	Konsentrasi transaksi per firma broker, seluruh pasar
Sumber data	`GET /api/broker-activity` → field `market` ← `broker_store/research.broker_daily` (data broker EOD asli IDX)
Rumus/Logika	Porsi transaksi per firma; bukan proksi
Periode/Window	Tanggal EOD terbaru (`market.date`)
Interaksi	—
Empty/Loading/Error	Skeleton / ErrorState "Gagal memuat struktur broker."

Komposisi Broker & Timeline (bagian bawah halaman)

Field	Isi
Nama card	Komposisi Broker; Timeline Aliran Dana
Komponen	`BrokerFlowCard` (BrokerFlowCard.tsx); `FlowTimelineCard` (FlowTimelineCard.tsx)

Field	Isi
Fungsi	Komposisi kategori broker; timeline skor satu emiten
Sumber data	`GET /api/broker-flow` (app.py:221); `GET /api/stocks/{code}/broker-activity` (app.py:378)
Rumus/Logika	Lihat Aliran Dana Emiten
Periode/Window	Lookback 60 sesi (timeline)
Interaksi	Timeline muncul setelah klik baris di tabel
Empty/Loading/Error	Lihat Aliran Dana Emiten

4. Data Flow (end-to-end)

```
Ingest broker EOD + harga + foreign → Postgres (broker_daily, latest_pit, stock_daily)
  → research.ic.build_factor_panel (lookback 60 hari, min_history 60)
  → research.broker_activity.select_factors / latest_factor_rows / composite_scores
  → broker_activity_snapshot (cache 1 jam) → get_broker_activity
  → GET /api/broker-activity?limit=25 → useBrokerActivity (300 s) → tabel + kartu
```

Snapshot yang sama juga dipakai Screener (`min_broker_score`) dan Ruang Keputusan.

5. Cara Pengambilan / Update Data

- **Precomputed sebagian**: data broker EOD diisi job `refresh_s2` (16:05) dan backfill broker 16:30 WIB (cli.py). IC diisi job `daily_ic` (Senin–Jumat 06:30 WIB).
- **On-demand**: snapshot dihitung saat request pertama setelah cache kosong.
- **Reset cache**: `POST /api/cache/clear`, dipanggil setiap pipeline refresh.
- **Refresh UI**: SWR 300 detik.
- **Idempotensi**: deterministik dari data tersimpan.
- **Fallback**: tidak tervalidasi → tidak ada skor palsu; UI menampilkan status.

6. Dependensi & Relasi Menu

- Bergantung pada Faktor & Kalibrasi (gate IC dan bobot).
- Ditampilkan juga di Screener dan Ruang Keputusan.
- Timeline membuka Aliran Dana Emiten.

7. Catatan Batasan & Edge Case

- Skor 0 tidak sama dengan "tidak ada data": emiten tanpa histori flow ≥ 21 hari tidak diberi skor.
- Tervalidasi = minimal satu faktor lolos gate; bila tidak, halaman menjelaskan alasannya (`reason`).
- Struktur broker pasar adalah data asli IDX, berlaku untuk seluruh pasar.

Aliran Dana Emiten (`/flow` + `/flow/[code]`)

1. Ringkasan Fungsi

Satu halaman untuk membaca aliran dana satu emiten: arah asing harian, timeline skor akumulasi, dan komposisi broker. Pengguna: analis yang sudah memilih emiten dan ingin melihat jejak aliran. Tanpa kode, halaman menampilkan pemilih dan empty state.

2. Route & Berkas

- URL indeks: `/flow` → `app/flow/page.tsx` (tanpa emiten)
- URL detail: `/flow/[code]` → `app/flow/[code]/page.tsx` [BELUM TERVERIFIKASI nama berkas]
- Workspace: `app/flow/FlowWorkspace.tsx`
- Komponen: `ForeignFlowCard`, `FlowTimelineCard`, `BrokerFlowCard` (components/)
- Hook: `useWatchlist()` (untuk saran cepat, hooks.ts); `useStockForeignFlow(code, 30, 10)` (hooks.ts:129); `useStockBrokerActivity(code, lookback)` (hooks.ts:227, 300 detik); `useBrokerFlow(20, 5)` (hooks.ts:117)

3. Penjelasan Per-Card / Per-Panel

Pemilih Emiten (StockPicker)

Field	Isi
Nama card	Pemilih emiten
Komponen	FlowWorkspace.tsx (`StockPicker`)
Fungsi	Input kode dan saran cepat (8 kode dari watchlist)
Sumber data	`useWatchlist()` → `GET /api/watchlist` (app.py:244)
Rumus/Logika	Filter kode berdasarkan teks; emiten aktif selalu ditawarkan
Periode/Window	—
Interaksi	Ketik kode + Enter/Lihat → `/flow/{KODE}`; chip cepat → link
Empty/Loading/Error	Saran kosong bila watchlist kosong

Arus Asing (ForeignFlowCard)

Field	Isi
Nama card	Arus Asing
Komponen	ForeignFlowCard.tsx (`code` prop)
Fungsi	Arah net asing harian, streak, flip terakhir, dan lama sejak flip
Sumber data	`GET /api/stocks/{code}/foreign-flow?days=30&peer_days=10` (app.py:397) → `get_stock_foreign_flow` (analytics.py:318); cache `stock_foreign_flow:{code}:{days}:{peer_days}:{peer_limit}`
Rumus/Logika	Arah = tanda net asing harian; streak = jumlah sesi berturut-turut searah; flip = pergantian tanda; peer = emiten sebanding
Periode/Window	30 sesi; peer 10 sesi
Interaksi	Hover pada titik peer; "Emiten ini" ditandai
Empty/Loading/Error	Pesan "Belum ada data aliran asing..." bila kosong; skeleton; error message

Timeline Aliran Dana (FlowTimelineCard)

Field	Isi
Nama card	Timeline Aliran Dana
Komponen	FlowTimelineCard.tsx
Fungsi	Skor akumulasi per tanggal dan median pasar sebagai pembanding
Sumber data	`GET /api/stocks/{code}/broker-activity?lookback=60` (app.py:378) → `get_stock_broker_activity` (analytics.py:2578); skor dari `broker_activity_snapshot` (analytics.py:2192)
Rumus/Logika	Skor 0–100 dari faktor aliran yang lolos gate IC; median = median skor pasar pada tanggal itu
Periode/Window	Lookback 60 sesi (default)
Interaksi	Tooltip per titik: tanggal, skor, median
Empty/Loading/Error	Pesan: butuh ≥ 2 sesi berskor dan faktor tervalidasi IC

Komposisi Broker (BrokerFlowCard)

Field	Isi
Nama card	Komposisi Broker
Komponen	BrokerFlowCard.tsx (`compact`)

Field	Isi
Fungsi	Porsi nilai transaksi per kategori broker: asing, lokal, BUMN
Sumber data	`GET /api/broker-flow?days=20&top_n=5` (app.py:221) → `get_broker_flow` (analytics.py:2055); tabel `research.broker_daily`
Rumus/Logika	Nilai per kategori dibagi total; kategori dari map kurasi `idx_scraper.broker_flow`
Periode/Window	20 sesi, top 5
Interaksi	Toggle peta broker (`showMap`)
Empty/Loading/Error	Pesan: isi lewat `idx serve` (job 16:30) atau `python -m scripts.backfill_broker_eod`

4. Data Flow (end-to-end)

```
Ingest EOD (foreign_flow, broker_daily) → Postgres
  → analytics.get_stock_foreign_flow / get_stock_broker_activity / get_broker_flow
  → GET /api/stocks/{code}/... → SWR → ForeignFlowCard, FlowTimelineCard, BrokerFlowCard
```

5. Cara Pengambilan / Update Data

- **On-demand** dengan cache in-process (600–3600 detik). Data sumber diisi job EOD.
- **Job**: `refresh_s2` 16:05 WIB, `refresh_s2_late` 16:20 WIB; broker EOD 16:30 WIB (cli.py).
- **Refresh UI**: SWR 30 detik (foreign, broker flow), 300 detik (broker activity per emiten).
- **Reset cache**: `POST /api/cache/clear`.
- **Idempotensi**: upsert per (tanggal, emiten/broker) [BELUM TERVERIFIKASI detail upsert].
- **Fallback**: pesan kosong; tidak ada sumber alternatif.

6. Dependensi & Relasi Menu

- Dibuka dari Foreign Flow, Aktivitas Broker, dan Halaman Detail Emiten.
- Skor berasal dari Faktor & Kalibrasi (IC gate).

7. Catatan Batasan & Edge Case

- Timeline butuh minimal 2 sesi berskor dan faktor tervalidasi; bila tidak, kartu kosong.
- Komposisi broker bersifat pasar, bukan per emiten.
- Data EOD: tidak real-time.

Backtest (`/backtest`)

1. Ringkasan Fungsi

Simulasi strategi di atas data historis untuk menguji apakah sebuah aturan entry/exit layak dipakai. Menjawab: "kalau aturan ini dijalankan sejak tanggal X dengan biaya nyata, hasilnya bagaimana dibanding beli-dan-tahan?" Pengguna: analis dan PM yang menguji ide sebelum dipakai. Halaman ini **hanya menghitung on-demand**; tidak ada hasil yang disimpan.

2. Route & Berkas

- URL: `/backtest` → `app/backtest/page.tsx`
- Hook: `useBacktestConfig()` (hooks.ts:410, refresh 300 detik) dan `runBacktest` (api.ts:118)
- Backend: simulation.py, endpoint app.py:693 (config) dan app.py:699 (run, POST)

3. Penjelasan Per-Card / Per-Panel

Form Konfigurasi (strategi, parameter, biaya, rebalance)

Field	Isi
Nama card	Konfigurasi Backtest
Komponen	backtest/page.tsx
Fungsi	Pilih strategi, parameter, rentang tanggal, daftar emiten, biaya, dan frekuensi rebalance
Sumber data	`GET /api/backtest/config` (app.py:693) → `simulation.get_config()` (simulation.py:296)
Rumus/Logika	Nilai default dan batas diambil dari konstanta: `DEFAULT_COSTS` (simulation.py:56), `COST_LIMITS`, `REBALANCE_OPTIONS` (daily/weekly/monthly, default `weekly`), `MAX_CODES=60`, `MAX_DAYS=756`
Periode/Window	Rentang tanggal dipilih pengguna, maksimum 756 hari
Interaksi	Ubah parameter → tombol jalankan → `POST /api/backtest/run`
Empty/Loading/Error	Skeleton saat config dimuat; pesan error bila gagal

Strategi yang Tersedia

Field	Isi
Nama card	Pilihan Strategi
Komponen	backtest/page.tsx
Fungsi	Tiga strategi: `ma_cross`, `momentum_top`, `rsi_oversold`, masing-masing dengan `ParamSpec` sendiri
Sumber data	`STRATEGIES` (simulation.py:234)
Rumus/Logika	`ma_cross`: sinyal dari persilangan rata-rata bergerak; `momentum_top`: peringkat return terkuat; `rsi_oversold`: entry saat RSI di bawah ambang. Detail parameter ada di `ParamSpec`
Periode/Window	Dikendalikan parameter setiap strategi
Interaksi	Pilih strategi → form parameter berubah
Empty/Loading/Error	Tidak relevan (daftar statis dari backend)

Hasil (Hasil)

Field	Isi
Nama card	Hasil
Komponen	backtest/page.tsx ~baris 514
Fungsi	Empat angka utama: total return neto, Alpha vs buy & hold, Max drawdown, Sharpe
Sumber data	`POST /api/backtest/run` (app.py:699) → `simulation.run_backtest` (simulation.py:546)
Rumus/Logika	Return dari equity curve neto biaya. Sharpe memakai `TRADING_DAYS_PER_YEAR=252` (simulation.py). Alpha = return strategi − return benchmark
Periode/Window	Rentang tanggal backtest
Interaksi	Tidak ada drill-down
Empty/Loading/Error	Muncul setelah run selesai; error ditampilkan bila backend gagal

Equity Curve

Field	Isi
Nama card	Equity Curve
Komponen	backtest/page.tsx ~baris 537 (Recharts)
Fungsi	Pertumbuhan nilai portofolio strategi vs benchmark dari waktu ke waktu
Sumber data	Field `equity_curve` pada respons `run_backtest`

Field	Isi
Rumus/Logika	Nilai portofolio harian setelah biaya
Periode/Window	Seluruh rentang backtest
Interaksi	Hover untuk nilai per tanggal
Empty/Loading/Error	Tidak tampil bila belum ada run

Perbandingan Metrik

Field	Isi
Nama card	Perbandingan Metrik
Komponen	backtest/page.tsx ~baris 549
Fungsi	Metrik kotor (`gross_metrics`) vs neto (`metrics`) dan benchmark (`benchmark`)
Sumber data	`run_backtest` → `metrics`, `gross_metrics`, `benchmark`
Rumus/Logika	Gross = sebelum biaya; neto = sesudah biaya; benchmark = equal-weighted buy & hold
Periode/Window	Seluruh rentang backtest
Interaksi	—
Empty/Loading/Error	Tidak tampil bila belum ada run

Dampak Biaya

Field	Isi
Nama card	Dampak Biaya
Komponen	backtest/page.tsx ~baris 561
Fungsi	Total biaya, komisi+pajak, slippage, dan cost drag
Sumber data	`run_backtest` → `costs`, `cost_impact`
Rumus/Logika	Komisi `commission_pct` 0,15%; pajak jual `sell_tax_pct` 0,1%; slippage `slippage_pct` 0,1%; partisipasi dibatasi `max_adv_pct` 2% dari ADV (`DEFAULT_COSTS`, simulation.py:56)
Periode/Window	Seluruh rentang backtest
Interaksi	—
Empty/Loading/Error	Tidak tampil bila belum ada run

Ledger Rebalance

Field	Isi
Nama card	Ledger Rebalance
Komponen	backtest/page.tsx ~baris 599
Fungsi	Catatan setiap transaksi per tanggal rebalance (beli/jual, jumlah, harga)
Sumber data	Field `ledger` pada `run_backtest`
Rumus/Logika	Dihasilkan saat rebalance sesuai frekuensi (daily/weekly/monthly)
Periode/Window	Per tanggal rebalance
Interaksi	Scroll tabel
Empty/Loading/Error	Kosong bila strategi tidak memicu transaksi

Cara Membaca Hasil

Field	Isi
Nama card	Cara membaca hasil
Komponen	backtest/page.tsx ~baris 616
Fungsi	Panduan interpretasi. Memuat catatan point-in-time: sinyal hanya memakai data yang sudah tersedia pada tanggal keputusan
Sumber data	Teks statis
Rumus/Logika	—
Periode/Window	—
Interaksi	—
Empty/Loading/Error	—

4. Data Flow (end-to-end)

```
Postgres (prices/OHLCV, stock_daily, index_quotes)
  → simulation.run_backtest (point-in-time, biaya, rebalance)
  → POST /api/backtest/run (compute-only, tanpa cache)
```

```
→ api.ts runBacktest → state lokal halaman
→ Recharts (equity curve) + tabel (metrik, biaya, ledger)
```

Config: `GET /api/backtest/config` → `useBacktestConfig` (SWR 300 detik) → form.

5. Cara Pengambilan / Update Data

- **On-demand**: setiap run menghitung ulang dari data harga tersimpan. Hasil tidak disimpan ke DB.
- **Job terjadwal**: tidak ada. Backtest tidak ikut pipeline refresh.
- **Refresh UI**: hasil tidak auto-refresh; pengguna menekan jalankan.
- **Idempotensi**: input sama + data harga sama → hasil sama (deterministik).
- **Fallback**: bila data harga kurang untuk rentang, backend mengembalikan error; UI menampilkan pesan.
- **Update data harga**: mengikuti pipeline EOD (lihat README); backtest hanya membaca.

6. Dependensi & Relasi Menu

- Bergantung pada data harga dari pipeline EOD (menu Dashboard, Pantau).
- Definisi faktor di strategi tidak terkait langsung dengan bobot Faktor & Kalibrasi; keduanya terpisah.

7. Catatan Batasan & Edge Case

- Maksimum 60 emiten dan 756 hari per run (`MAX_CODES`, `MAX_DAYS`).
- Benchmark selalu equal-weighted buy & hold dari emiten yang sama.
- Biaya default adalah asumsi; ubah di form bila kondisi berbeda.
- Hasil lampau bukan jaminan. Metrik dengan sampel kecil mudah bias.
- [BELUM TERVERIFIKASI] Nilai pasti parameter `ParamSpec` per strategi dan rumus Sharpe/drawdown di baris kode; cek `simulation.py` sebelum dikutip angka.

Dashboard (/)

1. Ringkasan Fungsi

Halaman pembuka: satu layar menjawab "bagaimana kondisi pasar hari ini?". Menampilkan IHSG, regime volatilitas, narasi pasar, ringkasan breadth, leaderboard likuiditas, sinyal teknikal, dan chart emiten terpilih. Pengguna: semua (trader, analis, PM). Dipakai saat membuka aplikasi / cek cepat kondisi sesi.

2. Route & Berkas

- URL: `/` → `app/page.tsx`
- Komponen: MarketOverview, MarketNarration, TopLeadersPanel, TopBrokersPanel, SignalsPanel, TechnicalChart, MarketRegimeCard, LastUpdated
- Hooks: `useMarketOverview`, `useMarketTradeSummary`, `useMarketRegime`, `useMarketLeaders`, `useTopBrokers`, `useSignals`, `useTechnical`, `useNarration` (hooks.ts)

3. Penjelasan Per-Card

Hero IHSG (Volume/Value/Emiten + Reguler/Non-Reguler)

Field	Isi
Nama card	IHSG · Composite (hero)
Komponen	`HeroIndex` / `SegmentBox` di `app/page.tsx:41`
Fungsi	Harga IHSG + perubahan, volume/value total, jumlah emiten aktif, rincian pasar reguler vs non-reguler
Sumber data	`GET /api/market/overview` (app.py:179) → `services.get_market_overview` (services.py:200) → tabel `index_quotes`; `GET /api/market/trade-summary` → `research.market_segment_daily`
Rumus/Logika	`index_quotes` close/change/percent; segment share = value segmen ÷ total (services.py:55 `_segment`, `_share`)
Periode/Window	Sesi terbaru; volume dalam lot (1 lot = 100 lembar)
Interaksi	Hover `InfoHint` untuk definisi pasar
Empty/Loading/Error	Bila `trade.date` null → teks "data sesi terbaru belum tersedia"

Suasana Pasar (Regime)

Field	Isi
Nama card	Suasana Pasar
Komponen	MarketRegimeCard.tsx
Fungsi	Status tren & volatilitas IHSG (ADX(14)+DI, realized vol 20 hari)
Sumber data	`GET /api/market/regime` (app.py:197) → `analytics.get_market_regime` (analytics.py:62) → `index_summary_daily`, fallback breadth `research.latest_pit`
Rumus/Logika	Regime dari ADX/DI + realized vol → label tren/ranging/transisi; klasifikasi di `research/regime.py`
Periode/Window	20 hari (vol), ADX 14
Interaksi	—
Empty/Loading/Error	Skeleton/Empty/Error via States.tsx

Narasi Pasar

Field	Isi
Nama card	Narasi Pasar
Komponen	MarketNarration.tsx
Fungsi	Ringkasan teks kondisi pasar (breadth, foreign, top movers)
Sumber data	`GET /api/market/narration` (app.py:512) → `analytics.get_market_narration` (analytics.py:1323) → `research.latest_pit`
Rumus/Logika	Agregasi qty value/volume/foreign_net per `trade_date` (analytics.py:1344-1365)
Periode/Window	Sesi terbaru
Interaksi	—
Empty/Loading/Error	Skeleton/Empty/Error

Ringkasan Pasar (Breadth)

Field	Isi
Nama card	Ringkasan Pasar
Komponen	MarketOverview.tsx
Fungsi	Jumlah naik/turun/stagnan, breadth

Field	Isi
Sumber data	`GET /api/market/overview` → `services.get_market_overview` → `index_quotes`
Rumus/Logika	Agregasi field `totals` di `get_market_overview`
Periode/Window	Sesi terbaru
Interaksi	—
Empty/Loading/Error	Skeleton/Empty/Error

Leaderboard (Volume / Value / Frekuensi)

Field	Isi
Nama card	Top Leader (3 varian)
Komponen	TopLeadersPanel.tsx (`metric="volume"/"value"/"frequency"`)
Fungsi	5 emiten teratas per metrik likuiditas
Sumber data	`GET /api/market/leaders?metric=` (app.py:207) → `services.get_market_leaders` (services.py:490) → `stock_summary_daily` / `stock_quotes`
Rumus/Logika	Realtime dari `stock_quotes` saat jam bursa, EOD dari `stock_summary_daily` (services.py:377 `_leaders_eod`, `:424` `_leaders_intraday`)
Periode/Window	Sesi terbaru
Interaksi	Klik emiten → set `selected` → chart di bawah
Empty/Loading/Error	Skeleton/Empty/Error

Top Broker

Field	Isi
Nama card	Top Broker
Komponen	TopBrokersPanel.tsx; logo kode broker lewat TickerLogo.tsx `basePath="/logos/brokers"`, monogram bila file tidak ada
Fungsi	Broker teraktif per nilai transaksi, dengan logo di samping kode broker
Sumber data	`GET /api/market/top-brokers` (app.py:215) → `services.get_top_brokers` (services.py:507) → `research.broker_daily`
Rumus/Logika	Rank value per `broker_code` pada `trade_date` terbaru
Periode/Window	EOD harian seluruh pasar
Interaksi	—
Empty/Loading/Error	Skeleton/Empty/Error

Trading Signals

Field	Isi
Nama card	Trading Signals
Komponen	SignalsPanel.tsx
Fungsi	Sinyal BUY/SELL/HOLD rule-based per emiten
Sumber data	`GET /api/signals` (app.py:304) → `services.get_signals` (services.py:1051) → `research.latest_pit`
Rumus/Logika	SMA20/50 + RSI(14) rule-based (SignalsPanel.tsx:128)
Periode/Window	SMA20/50, RSI 14, min 25 hari (services.py:1051)
Interaksi	Klik emiten → chart
Empty/Loading/Error	Skeleton/Empty/Error

Technical Chart

Field	Isi
Nama card	Technical Chart
Komponen	TechnicalChart.tsx
Fungsi	Close, MA20/50, Bollinger, RSI, MACD
Sumber data	`GET /api/stocks/{code}/technical` (app.py:325) → `services.get_technical_chart` (services.py:1098) → `stock_daily`, live bar dari `stock_quotes`
Rumus/Logika	OHLC historis + live bar di-merge (services.py:667 `_live_bar`, `:692` `_merge_live_bar`)
Periode/Window	~60 sesi
Interaksi	Tampil saat ada emiten terpilih
Empty/Loading/Error	Empty bila belum pilih emiten

4. Data Flow (end-to-end)

IDX/DataIDX → `cf_transport.py` → `live_capture.py` → PostgreSQL (`index_quotes`, `stock_quotes`, `index_summary_daily`, `stock_summary_daily`, `research.broker_daily`, `research.latest_pit`) → `analytics.py` / `services.py` → FastAPI (`/api/market/*`, `/api/signals`, `/api/stocks/{code}/technical`) →

SWR (30s) → `app/page.tsx` render.

5. Cara Pengambilan / Update Data

- **Kapan dibuat**: campuran — `overview/leaders` on-demand dari snapshot terbaru; `trade-summary` dari agregat EOD.
- **Job**: interval `idx` (60s, refresh index), `wl` (60s, watchlist/leaders). Cron EOD `refresh_s2` 16:05 mengisi snapshot harian. Lihat cli.py.
- **Refresh UI**: SWR 30s (overview/leaders/signals/technical), 60s (trade-summary/narration), 300s (regime).
- **Idempotensi**: upsert pada `trade_date/code`.
- **Fallback**: breadth dari `research.latest_pit` bila `index_summary_daily` kosong.

6. Dependensi & Relasi Menu

Feed ke hampir semua menu lain (klik emiten → `/stock/[code]`). Bergantung pada EOD pipeline yang juga mengisi Radar/Foreign/Aliran Dana.

7. Batasan & Edge Case

- Angka volume/value final hanya setelah pasar tutup.
- Data delayed ~5–15 menit (label UI).
- `stock_quotes` (live) bisa stale di luar jam bursa; leaderboard jatuh ke EOD.

Faktor & Kalibrasi (`/faktor`)

1. Ringkasan Fungsi

Menampilkan faktor mana yang terbukti memprediksi return (uji IC), bobot composite yang aktif, dan riwayat rekalibrasi. Menjawab: "faktor apa yang dipakai sistem sekarang dan seberapa kuat buktinya?" Pengguna: analis yang mengaudit model. Dibaca sebagai hasil run IC harian; bukan input interaktif.

2. Route & Berkas

- URL: `/faktor` → `app/faktor/page.tsx`
- Hook: `useFactorsOverview()` (hooks.ts:293, refresh 600 detik)
- Hook pendukung: `useRegimeHistory(90)` (hooks.ts:300)
- Backend: `GET /api/factors/overview` (app.py:593) → `get_factors_overview` (analytics.py:1859)

3. Penjelasan Per-Card / Per-Panel

Bobot Composite Aktif

Field	Isi
Nama card	Bobot Composite Aktif
Komponen	faktor/page.tsx ~baris 238
Fungsi	Bobot tiap faktor pada skor composite untuk horizon 10 sesi
Sumber data	`research.factor_ic_history` (kolom `weight`) → `get_factors_overview`; fallback konstanta `FACTOR_WEIGHTS` (vol_21d, turnover_21d, dist_52w_high)
Rumus/Logika	Bobot dari run IC terbaru untuk horizon 10. Bila tabel kosong, pakai fallback statis
Periode/Window	Run IC terbaru (`max(run_date)`)
Interaksi	—
Empty/Loading/Error	Fallback ke bobot statis bila belum ada run

Regime IHSG Historis

Field	Isi
Nama card	Regime IHSG Historis
Komponen	faktor/page.tsx ~baris 241
Fungsi	Riwayat regime pasar untuk konteks: faktor bisa bekerja beda di tiap regime
Sumber data	`GET /api/market/regime/history?days=90` (app.py:599) → `get_regime_history` (analytics.py:1940)
Rumus/Logika	Klasifikasi regime harian (lihat Dashboard)
Periode/Window	90 hari
Interaksi	—
Empty/Loading/Error	Placeholder bila data kosong

IC per Faktor — Horizon 5

Field	Isi
Nama card	IC per Faktor (Horizon 5)
Komponen	faktor/page.tsx ~baris 246
Fungsi	Mean IC, ICIR, t-stat, hit rate, jumlah hari, status eligible per faktor pada horizon 5 sesi
Sumber data	`research.factor_ic_history` (run terbaru) → `get_factors_overview`
Rumus/Logika	IC = Spearman rank correlation antara faktor dan return ke depan. Standard error Newey-West lag 5. Lolos bila |IC| ≥ 0,05 dan |ICIR| ≥ 0,5 (footer halaman)
Periode/Window	Horizon 5 sesi; `n_days` = jumlah hari observasi
Interaksi	—
Empty/Loading/Error	Pesan kosong bila belum ada run IC

IC per Faktor — Horizon 10

Field	Isi
Nama card	IC per Faktor (Horizon 10)
Komponen	faktor/page.tsx ~baris 249
Fungsi	Sama seperti horizon 5, pada horizon 10 sesi. Horizon ini dipakai untuk bobot composite
Sumber data	Sama dengan horizon 5
Rumus/Logika	Sama

Field	Isi
Periode/Window	Horizon 10 sesi
Interaksi	—
Empty/Loading/Error	Sama

Riwayat Rekalibrasi

Field	Isi
Nama card	Riwayat Rekalibrasi
Komponen	faktor/page.tsx ~baris 253
Fungsi	Bobot dan IC 12 run terakhir, untuk melihat stabilitas bobot
Sumber data	`research.factor_ic_history` (`history`, 12 run) → `get_factors_overview`
Rumus/Logika	Snapshot per `run_date`
Periode/Window	12 run terakhir
Interaksi	—
Empty/Loading/Error	Kosong bila belum ada histori

Registry Faktor

Field	Isi
Nama card	Registry Faktor
Komponen	faktor/page.tsx ~baris 257
Fungsi	Definisi tiap faktor (nama, arti, arah)
Sumber data	`FACTOR_DEFINITIONS` (analytics.py)
Rumus/Logika	Teks definisi statis
Periode/Window	—
Interaksi	—
Empty/Loading/Error	—

4. Data Flow (end-to-end)

```
Harga EOD (Postgres) → research.ic (build panel, IC harian)
  → research.factor_ic_history (per run_date)   [job daily_ic 06:30 WIB]
  → analytics.get_factors_overview (cache default)
  → GET /api/factors/overview → useFactorsOverview (600 s) → kartu faktor
```

5. Cara Pengambilan / Update Data

- **Precomputed**: IC dihitung job terjadwal `daily_ic` (Senin–Jumat 06:30 WIB, cli.py) dan disimpan ke `research.factor_ic_history`. Halaman hanya membaca.
- **Manual**: `idx ic` (disebut di pesan kosong analytics.py).
- **Cache**: respons di-cache in-process; `POST /api/cache/clear` (app.py:516) mengosongkannya.
- **Refresh UI**: SWR 600 detik.
- **Idempotensi**: satu baris per (run_date, faktor, horizon); run ulang menimpa.
- **Fallback**: bobot statis `FACTOR_WEIGHTS` bila tabel kosong.

6. Dependensi & Relasi Menu

- Bobot dan status eligible dipakai Aktivitas Broker, Ruang Keputusan, dan Screener.
- Konteks regime berasal dari Dashboard.

7. Catatan Batasan & Edge Case

- Faktor yang gagal ambang tetap tampil untuk transparansi tapi tidak diberi bobot.
- Data kurang dari histori minimum menghasilkan IC tidak stabil; lihat `n_days`.
- Bila job `daily_ic` gagal, halaman menampilkan run terakhir yang berhasil (tanggal `run_date` tidak berubah).
- [BELUM TERVERIFIKASI] Nama tabel `research.factor_ic_history` vs skema SQL di `sql/`; cocokkan sebelum dijadikan acuan migrasi.

Foreign Flow (`/foreign`)

1. Ringkasan Fungsi

Peringkat emiten dengan arus beli dan jual bersih asing dalam 20 sesi terakhir. Menjawab: "asing sedang masuk atau keluar di saham mana?" Pengguna: trader dan analis yang memantau posisi asing. Data EOD.

2. Route & Berkas

- URL: `/foreign` → `app/foreign/page.tsx`
- Hook: `useForeignFlow(days=20)` (hooks.ts:195, refresh 30 detik)
- Backend: `GET /api/foreign-flow?days=20` (app.py:392) → `get_foreign_flow` (analytics.py:250)

3. Penjelasan Per-Card / Per-Panel

Top Net Buy

Field	Isi
Nama card	Top Net Buy
Komponen	foreign/page.tsx ~baris 100–140
Fungsi	Emiten dengan akumulasi asing terbesar
Sumber data	`GET /api/foreign-flow` → `get_foreign_flow` (analytics.py:250); tabel `foreign_flow` / `latest_pit.foreign_net`
Rumus/Logika	Jumlah net asing per emiten selama 20 sesi, diurutkan menurun. Tiap baris menampilkan kode, persen, dan net (format ringkas)
Periode/Window	20 sesi (subtitle "Akumulasi asing terbesar")
Interaksi	Klik kode → Aliran Dana Emiten atau detail emiten
Empty/Loading/Error	Pesan kosong bila tidak ada data; skeleton saat memuat

Top Net Sell

Field	Isi
Nama card	Top Net Sell
Komponen	foreign/page.tsx ~baris 100–140
Fungsi	Emiten dengan distribusi asing terbesar
Sumber data	Sama dengan Top Net Buy
Rumus/Logika	Urutkan net asing menaik (paling negatif)
Periode/Window	20 sesi (subtitle "Distribusi asing terbesar")
Interaksi	Klik kode → detail
Empty/Loading/Error	Sama dengan Top Net Buy

4. Data Flow (end-to-end)

```
Ingest EOD asing (Postgres: foreign_flow, latest_pit.foreign_net)
  → analytics.get_foreign_flow (20 sesi)
  → GET /api/foreign-flow?days=20 → useForeignFlow (30 s) → dua kartu
```

5. Cara Pengambilan / Update Data

- **On-demand** dengan cache in-process; data dasar diisi job pipeline `refresh_s2` (16:05 WIB) dan `refresh_s2_late` (16:20 WIB).
- **Refresh UI**: SWR 30 detik; data tetap EOD sehingga tidak berubah intraday.
- **Reset cache**: `POST /api/cache/clear` (app.py:516).
- **Idempotensi**: net dihitung ulang dari data tersimpan; deterministik.
- **Fallback**: tidak ada sumber alternatif; kosong bila data asing belum diisi.

6. Dependensi & Relasi Menu

- Sumber untuk Radar Smart Money dan Sentimen.
- Detail per emiten di Aliran Dana Emiten dan Halaman Detail Emiten.

7. Catatan Batasan & Edge Case

- Hanya 20 sesi; ubah parameter `days` di backend bila perlu rentang lain.
- Data EOD: tidak mencerminkan transaksi intraday.
- [BELUM TERVERIFIKASI] Baris tabel dan kolom persis yang dibaca `get_foreign_flow` (analytics.py:250–317).

Framework Analisis Saham Multi-Agen

Dokumen ini adalah sumber kanonik peran agen untuk Market Labs. Sumber data berita ada di sumber-data.md.

Framework memecah tugas trading menjadi peran khusus. Setiap peran punya input, output, dan batas kewenangan yang jelas.

1. Tim Analis

Analis bekerja bersamaan dengan tools masing-masing. Debat riset dimulai setelah seluruh laporan selesai.

Peran	Tugas	Input utama	Output
Analis Fundamental	Mengevaluasi laporan keuangan dan metrik kinerja, mengidentifikasi nilai intrinsik dan red flag	Laporan keuangan (idx.co.id), rasio emiten	Laporan fundamental: nilai intrinsik, red flag
Analis Sentimen	Mengagregasi headline menjadi satu pembacaan sentimen jangka pendek	Headline berita, sentimen sosial	Skor sentimen harian
Analis Berita	Memantau berita global dan makro, menginterpretasikan dampaknya ke pasar	Berita market, data makro (BI rate, Fed, inflasi)	Ringkasan dampak peristiwa
Analis Teknikal	Mendeteksi pola trading dan memprediksi pergerakan harga	OHLCV, MACD, RSI	Sinyal teknikal dan level kunci

2. Tim Peneliti (Researcher Team)

- Peneliti bullish dan bearish menilai secara kritis insight Tim Analis.
- Debat terstruktur menyeimbangkan potensi keuntungan dengan risiko.
- Output: rangkuman argumen bullish dan bearish dengan bukti masing-masing.

3. Agen Trader

- Menyusun laporan dari Tim Analis dan Tim Peneliti.
- Menentukan timing dan magnitude transaksi.
- **Batas**: hanya mengusulkan transaksi. Keputusan akhir ada di Portfolio Manager.

4. Manajemen Risiko dan Manajer Portofolio

- **Tim Manajemen Risiko**: terus menilai risiko portofolio (volatilitas, likuiditas, faktor risiko lain). Menyesuaikan strategi dan memberi laporan penilaian ke Portfolio Manager.
- **Manajer Portofolio**: menyetujui atau menolak proposal transaksi.
- Jika disetujui, order dikirim ke bursa simulasi dan dieksekusi.

5. Tim Engineering

Berjalan paralel dengan Tim Analis dan bekerja terus di background. Tugasnya menjaga data dan eksekusi tetap andal.

Peran	Tugas
Data Engineer / Pipeline Agent	Menjaga ketersediaan dan kualitas data real-time (harga, berita, on-chain, sentimen sosial). Memastikan tidak ada data telat, hilang, atau korup sebelum masuk analisis.
Execution Engineer / Infra Agent	Koneksi API, latency monitoring, reliability eksekusi order. Menerjemahkan keputusan Trader Agent.
System Monitoring Agent	Mengawasi uptime, error rate, dan anomali output antar-agen. Memberi alert sebelum output tidak masuk akal sampai ke Portfolio Manager.

6. Tim Frontend

Bertanggung jawab atas kualitas visual dan teknis antarmuka. Target: UI yang terasa dirancang dengan intensi, bukan template generik yang mudah dikenali sebagai buatan AI.

Hindari: layout kaku-simetris, gradient ungu-biru klise, kartu dengan shadow seragam, spacing "aman" tapi hambar, hero section generik, ikon dari satu library tanpa penyesuaian, dan copy placeholder yang tidak diganti.

Peran	Tugas
UI/UX Craft Agent	Menentukan arah desain spesifik per proyek: tipografi, palet, layout sesuai konteks produk.
Interaction & Motion Agent	Interaksi dan transisi yang punya tujuan. State hover, loading, dan feedback harus terasa hidup. Bukan animasi dekoratif.
Code Quality Agent	Komponen modern yang reusable, accessible, responsive. Tidak ada kode tempelan hasil copy-paste.
Review/QA Agent	Audit hasil akhir dengan checklist "no AI slop": generik? elemen asal tempel? konsisten dengan identitas visual?

7. Tim Trainer

Menjaga dan meningkatkan kualitas output seluruh agen (trading dan frontend) lewat iterasi, evaluasi, dan kalibrasi berkelanjutan. Tidak dideploy sekali lalu dibiarkan.

Peran	Tugas
Evaluation Agent	Menyusun dan menjalankan test case dan benchmark untuk mengukur akurasi analisis dan konsistensi desain. Membandingkan hasil dari waktu ke waktu.
Feedback Loop Agent	Mengumpulkan feedback dari hasil nyata (performa trading, review desain, komplain user). Menerjemahkannya jadi penyesuaian prompt atau parameter.
Calibration Agent	Menyesuaikan arah strategi atau gaya (risk appetite trader, konsistensi desain) berdasarkan tren performa, agar sistem tidak drift dari tujuan awal.

Catatan Status

- Dokumen ini mendeskripsikan **peran**, bukan status implementasi. Status implementasi per fitur ada di README.md.
- Peran yang belum ada implementasinya di kode harus ditandai `[BELUM TERVERIFIKASI]` sampai dibuktikan.

Halaman Detail Emiten (`/stock/[code]`)

1. Ringkasan Fungsi

Satu halaman lengkap untuk satu emiten: grafik teknikal, sinyal, kepemilikan, arus asing, aliran dana, broker, event study, dan riwayat harga. Pengguna: analis yang sudah memilih emiten dari menu lain. Navigasi prev/next mengikuti urutan watchlist.

2. Route & Berkas

- URL: `/stock/[code]` → `app/stock/[code]/page.tsx`
- Komponen (diimpor di page.tsx baris 7–18): `TechnicalChart`, `HistoryTable`, `SignalsPanel`, `BrokerSummary`, `ForeignFlowCard`, `FlowTimelineCard`, `BrokerFlowCard`, `BrokerActivityCard`, `RegimeBanner`, `EventStudyCard`, `SmartMoneyBanner`, `OwnershipCard`
- Hook: `useWatchlist()` (navigasi prev/next, hooks.ts)

3. Penjelasan Per-Card / Per-Panel

Header & Navigasi Emiten

Field	Isi
Nama card	Header kode + tombol prev/next
Komponen	stock/[code]/page.tsx baris 30–80
Fungsi	Menampilkan kode dan pindah ke emiten sebelum/sesudah dalam watchlist
Sumber data	`GET /api/watchlist` (app.py:244)
Rumus/Logika	Urutan array watchlist; prev/next = indeks ± 1
Periode/Window	—
Interaksi	Tombol prev/next → `/stock/{kode}`
Empty/Loading/Error	Tombol nonaktif bila tidak ada tetangga

Grafik Teknikal (TechnicalChart)

Field	Isi
Nama card	Grafik Teknikal
Komponen	`components/TechnicalChart.tsx` [BELUM TERVERIFIKASI baris]
Fungsi	Candlestick dengan indikator (SMA, MACD, RSI)
Sumber data	`GET /api/stocks/{code}/technical` (app.py:325) → `services.get_technical_chart` (services.py:1098); hook `useTechnical` refresh 30 detik
Rumus/Logika	Indikator dihitung di backend dari OHLC
Periode/Window	Ditentukan backend [BELUM TERVERIFIKASI]
Interaksi	Hover per candle
Empty/Loading/Error	Skeleton; pesan error

Sinyal (SignalsPanel)

Field	Isi
Nama card	Sinyal
Komponen	`components/SignalsPanel.tsx` [BELUM TERVERIFIKASI baris]
Fungsi	Sinyal BUY/SELL untuk emiten ini
Sumber data	`GET /api/signals?min_days=…` (app.py:304); hook `useSignals`, 30 detik
Rumus/Logika	Lihat Jejak Sinyal
Periode/Window	Parameter `min_days`
Interaksi	—
Empty/Loading/Error	Pesan kosong

Kepemilikan (OwnershipCard)

Field	Isi
Nama card	Kepemilikan
Komponen	`components/OwnershipCard.tsx` [BELUM TERVERIFIKASI baris]
Fungsi	Komposisi pemegang saham
Sumber data	`GET /api/stocks/{code}/ownership` (app.py:439) → `get_stock_ownership` (analytics.py:1004); cache 600 detik; hook 3600 detik
Rumus/Logika	Dari data kepemilikan tersimpan (tabel `research.ownership`)

Field	Isi
Periode/Window	Periode laporan kepemilikan
Interaksi	—
Empty/Loading/Error	Pesan kosong

Arus Asing, Timeline, Broker

Field	Isi
Nama card	Arus Asing; Timeline Aliran Dana; Komposisi Broker; Aktivitas Broker
Komponen	`ForeignFlowCard`, `FlowTimelineCard`, `BrokerFlowCard`, `BrokerActivityCard`
Fungsi	Lihat Aliran Dana Emiten dan Aktivitas Broker
Sumber data	`/api/stocks/{code}/foreign-flow`, `/broker-activity`, `/api/broker-flow`
Rumus/Logika	Lihat menu terkait
Periode/Window	Lihat menu terkait
Interaksi	—
Empty/Loading/Error	Lihat menu terkait

Broker Summary (BrokerSummary)

Field	Isi
Nama card	Ringkasan Broker
Komponen	`components/BrokerSummary.tsx` [BELUM TERVERIFIKASI baris]
Fungsi	Broker teratas untuk emiten ini
Sumber data	`GET /api/stocks/{code}/brokers` (app.py:336); hook refresh 30 detik
Rumus/Logika	Dari `broker_summary` per emiten
Periode/Window	Tanggal EOD
Interaksi	—
Empty/Loading/Error	Pesan kosong

Event Study (EventStudyCard)

Field	Isi
Nama card	Event Study
Komponen	`components/EventStudyCard.tsx` [BELUM TERVERIFIKASI baris]
Fungsi	Reaksi harga di sekitar event (dividen, aksi korporasi, pengumuman)
Sumber data	`GET /api/stocks/{code}/events` (app.py:584) → `get_stock_events` (analytics.py:1809); cache 6 jam
Rumus/Logika	Return abnormal sekitar tanggal event [BELUM TERVERIFIKASI window]
Periode/Window	Window event
Interaksi	—
Empty/Loading/Error	Pesan kosong

Riwayat Harga (HistoryTable)

Field	Isi
Nama card	Riwayat Harga
Komponen	`components/HistoryTable.tsx` [BELUM TERVERIFIKASI baris]
Fungsi	Tabel OHLCV harian
Sumber data	`GET /api/stocks/{code}/history?limit=…` (app.py:314); hook refresh 30 detik
Rumus/Logika	Data mentah dari tabel harga
Periode/Window	Parameter `limit`
Interaksi	Scroll
Empty/Loading/Error	Pesan kosong

4. Data Flow (end-to-end)

```
Postgres (stock_daily, stock_quotes, foreign_flow, broker_summary, research.*)
  → endpoint /api/stocks/{code}/* (app.py)
  → SWR hook per kartu (lib/hooks.ts)
  → komponen kartu di halaman stock/[code]
```

5. Cara Pengambilan / Update Data

- **Campuran**: harga dan broker dibaca langsung dari DB (refresh 30 detik); skor dan event di-cache.
- **Job**: lihat README bagian scheduler.
- **Reset cache**: `POST /api/cache/clear`.
- **Fallback**: kartu kosong menampilkan pesannya sendiri; kartu lain tetap jalan.

6. Dependensi & Relasi Menu

- Menggabungkan Sinyal/Jejak Sinyal, Foreign Flow, Aliran Dana Emiten, Aktivitas Broker, Radar Smart Money.

7. Catatan Batasan & Edge Case

- Watchlist kosong → navigasi prev/next tidak tersedia.
- Banyak kartu memakai data EOD; data intraday hanya tersedia bila capture berjalan.
- [BELUM TERVERIFIKASI] Nama berkas dan baris untuk komponen bertanda; cek sebelum dikutip.

Jejak Sinyal (`/jejak-sinyal`)

1. Ringkasan Fungsi

Rapor kejujuran sinyal BUY/SELL: hit-rate, return setelah sinyal, selisih vs pasar equal-weight, dipecah per regime IHSG. Menjawab "kalau sinyal aplikasi ini diikuti, apakah benar-benar menang?". Pengguna: analis/PM yang menilai kredibilitas sinyal.

2. Route & Berkas

- URL: `/jejak-sinyal` → `app/jejak-sinyal/page.tsx` (233 baris)
- Komponen: `SummaryStrip`, `HorizonPills`, `StatTable` (di file page), LastUpdated, States
- Hooks: `useSignalTrack()` (600 s) (hooks.ts)

3. Penjelasan Per-Card

Ringkasan (SummaryStrip)

Field	Isi
Nama card	Ringkasan strip (jumlah sinyal, periode, tanggal data terakhir)
Komponen	`SummaryStrip` jejak-sinyal/page.tsx
Fungsi	Gambaran cepat cakupan track record
Sumber data	`GET /api/signals/track` (app.py:563) → `analytics.signal_track` (analytics.py:2023)
Rumus/Logika	Agregasi jumlah & rentang tanggal dari `signal_log`
Periode/Window	Seluruh riwayat `signal_log`
Interaksi	—
Empty/Loading/Error	Skeleton

Kinerja per Sinyal

Field	Isi
Nama card	Kinerja per Sinyal
Komponen	jejak-sinyal/page.tsx:157 + `StatTable`
Fungsi	Rata-rata return, excess return vs equal-weight, t-stat (Newey-West lag 5), max runup/drawdown per sinyal
Sumber data	`analytics.signal_track` → `research.signal_log` (2024, 2034) + `research.latest_pit` untuk harga
Rumus/Logika	Entry = close T+1; excess = return sinyal − return pasar equal-weight window sama; t-stat Newey-West lag 5
Periode/Window	Horizon 5 / 10 / 21 sesi (pill `HorizonPills`)
Interaksi	Pilih horizon (5h/10h/21h)
Empty/Loading/Error	Skeleton; tabel kosong bila belum ada sinyal

Kinerja per Regime IHSG

Field	Isi
Nama card	Kinerja per Regime IHSG
Komponen	jejak-sinyal/page.tsx:170
Fungsi	Membandingkan kinerja sinyal saat trending / ranging / transisi
Sumber data	`analytics.signal_track` → `research.signal_log` join `research.regime_daily`
Rumus/Logika	Sinyal dikelompokkan berdasarkan `regime` pada tanggal sinyal
Periode/Window	Horizon mengikuti pill aktif
Interaksi	Horizon bersama dengan kartu sebelumnya
Empty/Loading/Error	Skeleton / kosong bila `regime_daily` belum terisi

Sinyal Terbaru

Field	Isi
Nama card	Sinyal Terbaru
Komponen	jejak-sinyal/page.tsx:180 (tabel)
Fungsi	Daftar sinyal terbaru + return realisasi 5h/10h/21h
Sumber data	`analytics.signal_track` → `research.signal_log`
Rumus/Logika	Return terealisasi dari close entry; kosong bila horizon belum selesai
Periode/Window	5 / 10 / 21 sesi

Field	Isi
Interaksi	—
Empty/Loading/Error	EmptyState "Belum ada sinyal tercatat."

4. Data Flow (end-to-end)

```
Pipeline sinyal (services.get_signals) → research.signal_log
  + research.latest_pit (harga) + research.regime_daily
  → analytics.signal_track → GET /api/signals/track → SWR (600 s) → 4 kartu
```

5. Cara Pengambilan / Update Data

- **Perhitungan**: on-demand dari `signal_log`; hasil di-cache (research cache 3600 s, `_RESEARCH_TTL`).
- **Pencatatan sinyal**: `signal_log` diisi pipeline sinyal; hasil horizon terisi seiring waktu.
- **Job**: ikut pipeline EOD; cache dibersihkan oleh `POST /api/cache/clear` tiap pipeline refresh.
- **Refresh UI**: 600 s.
- **Idempotensi**: [BELUM TERVERIFIKASI — cek unique key `signal_log`].

6. Dependensi & Relasi Menu

- Input: sinyal dari Pantau/Screener pipeline; regime dari Faktor & Kalibrasi.
- Menilai hasil Rekomendasi Beli dan Backtest pada sisi kejujuran sinyal.

7. Catatan Batasan & Edge Case

- Kolom return kosong = horizon belum selesai (bukan nol).
- Sampel kecil per regime → t-stat tidak stabil; baca bersama jumlah sinyal.
- Hanya sinyal yang tercatat di `signal_log` yang dinilai; sinyal belum tercatat tidak ikut.

Pantau (`/pantau`)

1. Ringkasan Fungsi

Pusat watchlist + notifikasi. Menjawab "saham apa yang saya pantau, dan kabari saya kalau ada apa-apa". Pengguna: trader/analis yang memantau emiten pilihan. Dipakai harian untuk kelola watchlist, pasang batas, dan terima alert Telegram.

2. Route & Berkas

- URL: `/pantau` → `app/pantau/page.tsx`
- Komponen: WatchlistTable, TechnicalChart, SmartMoneyWatchCard, AlertRules, TelegramStatus
- Hooks: `useWatchlist`, `useSignals`, `useTechnical`, `useSmartMoneyWatchlist`, `useAlerts`, `useAddToWatchlist`, `useRemoveFromWatchlist`, `useCreateAlert`, `useDeleteAlert` (hooks.ts)

3. Penjelasan Per-Card

Watchlist & Tabel Emiten

Field	Isi
Nama card	Watchlist (tabel, klik baris → pilih emiten)
Komponen	WatchlistTable.tsx
Fungsi	Daftar emiten pantauan + harga/perubahan; baris terpilih menggerakkan chart di bawah
Sumber data	`GET /api/watchlist` (app.py:244) → `services.get_watchlist` (services.py:334); tambah `POST /api/watchlist` (249), ubah `PUT /api/watchlist` (260), hapus `DELETE /api/watchlist` (274)
Rumus/Logika	Gabungan snapshot harga (`stock_quotes`) per kode watchlist
Periode/Window	Sesi terbaru; refresh SWR 30 detik
Interaksi	Klik baris → `onSelect` → chart; tombol tambah/hapus emiten
Empty/Loading/Error	Empty state bila watchlist kosong

Chart Teknikal Emiten Terpilih

Field	Isi
Nama card	Grafik teknikal (mengikuti emiten terpilih)
Komponen	TechnicalChart.tsx
Fungsi	Candlestick + overlay indikator (SMA/RSI/MACD per mode)
Sumber data	`GET /api/stocks/{code}/technical` (app.py:325) → `services.get_technical_chart` (services.py:1098); OHLC via `GET /api/stocks/{code}/history` (314) → `services.get_price_history` (services.py:715)
Rumus/Logika	Indikator dihitung di `services` dari frame OHLC (`prices_adjusted`)
Periode/Window	120 sesi (default history); refresh 30 detik
Interaksi	Ganti emiten lewat tabel; hover tooltip
Empty/Loading/Error	Placeholder bila belum ada emiten terpilih

Kartu Smart Money Watchlist

Field	Isi
Nama card	Verdict Smart Money (verdicts board)
Komponen	SmartMoneyWatchCard.tsx
Fungsi	Ringkasan verdict akumulasi/distribusi untuk emiten yang dipantau
Sumber data	`GET /api/smart-money/verdicts` (app.py:478) → `analytics.get_smart_money_verdicts` (analytics.py:924) → `research.latest_pit`
Rumus/Logika	Verdict dari net asing + pola aliran terbaru
Periode/Window	Jendela aliran terbaru (5 sesi default)
Interaksi	Tautan ke detail emiten
Empty/Loading/Error	Empty bila belum ada verdict

Aturan Alert

Field	Isi
Nama card	Aturan Notifikasi
Komponen	AlertRules.tsx
Fungsi	Bangun/kelola aturan harga/RSI/volume; notifikasi sekali per persilangan
Sumber data	`GET /api/alerts` (app.py:650); `POST /api/alerts` (656); `DELETE /api/alerts/{rule_id}` (666); `POST /api/alerts/test` (674)

Field	Isi
Rumus/Logika	Dievaluasi tiap kali data EOD baru masuk; state "tersimpan aman" mencegah spam
Periode/Window	Evaluasi per data EOD masuk
Interaksi	Form tambah aturan, hapus, uji kirim
Empty/Loading/Error	Pesan bila Telegram belum dikonfigurasi

Status Telegram

Field	Isi
Nama card	Status Notifikasi Telegram
Komponen	TelegramStatus.tsx
Fungsi	Kesiapan saluran notifikasi (configured/aktif)
Sumber data	`GET /api/health` (app.py:174) / status alert
Rumus/Logika	Cek konfigurasi bot Telegram (nama variabel env saja)
Periode/Window	Real-time
Interaksi	—
Empty/Loading/Error	Peringatan bila token/chat belum diset

4. Data Flow (end-to-end)

```
IDX (EOD / intraday) → cf_transport / ingest → Postgres (stock_quotes, prices_pit)
   → alert job (interval 300s, cli cmd_serve) evaluasi aturan → Telegram
   → API /api/watchlist + /api/alerts → SWR (useWatchlist 30s, useAlerts 30s)
   → WatchlistTable / AlertRules / TelegramStatus
```

5. Cara Pengambilan / Update Data

- **Watchlist & chart**: on-demand saat request; snapshot harga dari sesi terbaru.
- **Alert**: job `alerts` interval 300 detik di `cmd_serve` (cli.py:1037), hanya Telegram.
- **Refresh UI**: SWR 30 detik (watchlist, signals, technical, alerts).
- **Idempotensi**: evaluasi aturan sekali per persilangan (state per aturan).
- **Fallback**: bila sumber resmi gagal, ingest memakai fallback Yahoo Finance (lihat README indeks).

6. Dependensi & Relasi Menu

- Bergantung pada snapshot `stock_quotes` (dari job refresh EOD) dan `prices_adjusted`.
- Menyediakan emiten terpilih ke Jejak Sinyal, Ruang Keputusan, dan halaman detail `/stock/[code]`.

7. Catatan Batasan & Edge Case

- Alert hanya bunyi setelah data EOD baru masuk; di luar jam bursa tidak ada evaluasi baru.
- Emiten baru perlu data OHLC minimal sebelum indikator bisa dihitung.
- Kolom skor broker/smart-money hanya muncul bila run IC tersedia.

Radar Smart Money (`/radar` + `/radar/[code]`)

1. Ringkasan Fungsi

Mendeteksi emiten dengan aliran asing dominan: akumulasi (net buy asing terbesar) dan distribusi (net sell asing terbesar) dalam N sesi. Dilengkapi papan pola, track record, dan detail per emiten. Pengguna: trader dan analis yang mencari jejak "uang pintar" (asing sebagai proksi). Data EOD, bukan real-time.

2. Route & Berkas

- URL: `/radar` → `app/radar/page.tsx`
- URL detail: `/radar/[code]` → `app/radar/[code]/page.tsx`
- Komponen: `SmartMoneyBanner`, `SmartMoneyPatternsBoardCard`, `SmartMoneyTrackRecordCard` (components/)
- Hook: `useSmartMoneyRadar(10)` (hooks.ts:423, 300 detik); `useStockSmartMoney(code)` (hooks.ts:435, 300 detik); `useSmartMoneyPatternsBoard` (hooks.ts:458, 1800 detik); `useSmartMoneyTrackRecord` (hooks.ts:446, 3600 detik)
- Backend: `GET /api/smart-money/radar` (app.py:413) → `get_smart_money_radar` (analytics.py:497); `GET /api/stocks/{code}/smart-money` (app.py:427) → `get_stock_smart_money` (analytics.py:585); `GET /api/smart-money/track-record` (app.py:452); `GET /api/smart-money/patterns` (app.py:464); `GET /api/smart-money/verdicts` (app.py:478)

3. Penjelasan Per-Card / Per-Panel

Akumulasi

Field	Isi
Nama card	Akumulasi
Komponen	radar/page.tsx ~baris 150–205
Fungsi	Daftar emiten dengan net buy asing terbesar
Sumber data	`GET /api/smart-money/radar?days=10` → `get_smart_money_radar` (analytics.py:497); sumber baris asing `latest_pit.foreign_net` / `foreign_flow`
Rumus/Logika	Urutkan net asing (rupiah = volume asing × close) menurun; ambil teratas
Periode/Window	10 sesi (subtitle "N sesi")
Interaksi	Klik emiten → `/radar/[code]`
Empty/Loading/Error	Pesan kosong bila tidak ada aliran asing; skeleton saat memuat

Distribusi

Field	Isi
Nama card	Distribusi
Komponen	radar/page.tsx ~baris 150–205
Fungsi	Daftar emiten dengan net sell asing terbesar
Sumber data	Sama dengan Akumulasi
Rumus/Logika	Urutkan net asing menaik (paling negatif)
Periode/Window	10 sesi
Interaksi	Klik emiten → `/radar/[code]`
Empty/Loading/Error	Sama dengan Akumulasi

Catatan Footer

Field	Isi
Nama card	Catatan data (footer)
Komponen	radar/page.tsx ~baris 150–205
Fungsi	Menjelaskan keterlambatan EOD (±5–15 menit), definisi rupiah, dan jumlah emiten yang dipindai
Sumber data	Teks statis + field `scanned` dari respons radar
Rumus/Logika	Rupiah = volume asing × close
Periode/Window	—
Interaksi	—
Empty/Loading/Error	—

Papan Pola Smart Money

Field	Isi
Nama card	Papan Pola (`SmartMoneyPatternsBoardCard`)
Komponen	`components/SmartMoneyPatternsBoardCard.tsx` [BELUM TERVERIFIKASI baris]
Fungsi	Pola aliran (mis. akumulasi berkelanjutan, flip) per emiten
Sumber data	`GET /api/smart-money/patterns` (app.py:464) → `get_smart_money_patterns_board` (analytics.py:556); cache `smart_money_patterns_board` 1800 detik
Rumus/Logika	Dari deret net asing harian; detail di fungsi backend
Periode/Window	Ditentukan backend
Interaksi	Klik emiten → detail
Empty/Loading/Error	Pesan kosong bila data belum cukup

Track Record Smart Money

Field	Isi
Nama card	Track Record (`SmartMoneyTrackRecordCard`)
Komponen	`components/SmartMoneyTrackRecordCard.tsx` [BELUM TERVERIFIKASI baris]
Fungsi	Seberapa baik sinyal aliran asing historis memprediksi return
Sumber data	`GET /api/smart-money/track-record` (app.py:452) → `get_smart_money_track_record` (analytics.py:845); cache 3600 detik
Rumus/Logika	Evaluasi sinyal pada horizon tetap; dibandingkan dengan pasar
Periode/Window	Histori sinyal
Interaksi	—
Empty/Loading/Error	Pesan kosong bila histori kurang

Smart Money per Emiten (`/radar/[code]`)

Field	Isi
Nama card	Banner Smart Money + Track Record emiten
Komponen	radar/[code]/page.tsx; `SmartMoneyBanner`
Fungsi	Ringkasan arah aliran asing satu emiten + posisi di radar
Sumber data	`GET /api/stocks/{code}/smart-money` → `get_stock_smart_money` (analytics.py:585); cache `smart_money_stock:{code}` 1800 detik
Rumus/Logika	Dari arus asing emiten; detail di fungsi backend
Periode/Window	Ditentukan backend
Interaksi	Tombol kembali ke `/radar`
Empty/Loading/Error	Pesan kosong bila kode tidak punya data

4. Data Flow (end-to-end)

```
Broker/foreign EOD → Postgres (foreign_flow, latest_pit.foreign_net)
  → analytics.get_smart_money_radar / get_smart_money_patterns_board / get_smart_money_track_record
  → GET /api/smart-money/* → SWR (300 s / 1800 s / 3600 s) → kartu radar
  → klik → /radar/[code] → GET /api/stocks/{code}/smart-money → SmartMoneyBanner
```

5. Cara Pengambilan / Update Data

- **Precomputed vs on-demand**: hasil dihitung saat request, lalu di-cache in-process (1800–3600 detik). Sumber data EOD diisi oleh job pipeline `refresh_s2` (16:05 WIB) dan `refresh_s2_late` (16:20 WIB) (cli.py).
- **Refresh UI**: SWR 300 detik (radar, detail) dan 1800/3600 detik (papan dan track record).
- **Reset cache**: `POST /api/cache/clear` (app.py:516), dipanggil setiap job pipeline.
- **Idempotensi**: perhitungan murni dari data tersimpan; run ulang menghasilkan angka sama.
- **Fallback**: bila data asing kosong, kartu menampilkan empty state; tidak ada sumber alternatif.

6. Dependensi & Relasi Menu

- Memakai data Foreign Flow dan Aliran Dana Emiten.
- Hasil ringkas ikut tampil di Halaman Detail Emiten.
- Verdict papan (`/api/smart-money/verdicts`) dipakai Screener / watchlist [BELUM TERVERIFIKASI konsumen tepat].

7. Catatan Batasan & Edge Case

- Data EOD: sinyal baru tersedia setelah bursa tutup.
- Rupiah dihitung dari volume asing × close, bukan nilai transaksi resmi IDX.

- Track record butuh histori cukup; sampel kecil membuat hasil tidak stabil.
- Cache 1800–3600 detik: data baru tampil setelah cache habis atau dibersihkan.

Rekomendasi Beli (`/rekomendasi`)

1. Ringkasan Fungsi

Papan kandidat beli se-pasar dengan level masuk, stop, target, dan R/R, diurutkan dari bukti terkuat. Menjawab "emiten mana yang lolos gate kalkulasi hari ini, dan seberapa terbukti hasilnya". Pengguna: trader/analis. Dipakai pagi/sore setelah data EOD masuk.

2. Route & Berkas

- URL: `/rekomendasi` → `app/rekomendasi/page.tsx` (401 baris)
- Komponen: `LayerStrip`, `CandidateCard` (di file page), RegimeBanner, States, LastUpdated
- Hooks: `useRecommendations(limit, minGrade)` (300 s), `useRecommendationTrack()` (3600 s) (hooks.ts)

3. Penjelasan Per-Card

Papan Kandidat

Field	Isi
Nama card	Papan Kandidat
Komponen	rekomendasi/page.tsx:330 (Card) + `CandidateCard`
Fungsi	Daftar kandidat beli: kode, grade A/B/C, entry, stop, target, R/R, alasan
Sumber data	`GET /api/recommendations` (app.py:341) → `analytics.recommendations` (analytics.py:2798) → `research.latest_pit`, `research.recommendation_log` (2776–2791)
Rumus/Logika	Gate: kandidat wajib punya stop yang dapat dihitung. Grade = peringkat relatif dalam pool hari itu: A 2% · B 10% · C 25% teratas (bukan probabilitas). Skor hanya dari lapisan lolos walk-forward
Periode/Window	Sesi terbaru; `limit` 10/20/50/100 baris
Interaksi	Filter grade (A saja / B ke atas / C ke atas); pilih jumlah baris
Empty/Loading/Error	Skeleton; EmptyState khusus grade A ("turunkan filter ke B/C"); ErrorState pesan backend/DB

Lapisan (LayerStrip)

Field	Isi
Nama card	Strip lapisan skor (di atas papan)
Komponen	`LayerStrip` di rekomendasi/page.tsx
Fungsi	Menampilkan lapisan yang aktif/tidak lolos gate
Sumber data	Field `layers` dari `GET /api/recommendations`
Rumus/Logika	Lapisan yang belum lolos gate tidak ikut menimbang skor
Periode/Window	Sesi terbaru
Interaksi	—
Empty/Loading/Error	Tidak tampil bila data belum ada

Track Record Kandidat

Field	Isi
Nama card	Track Record Kandidat
Komponen	rekomendasi/page.tsx:397
Fungsi	Hit-rate dan return kandidat historis ("diukur, bukan diklaim")
Sumber data	`GET /api/recommendations/track` (app.py:355) → `analytics.recommendation_track` (analytics.py:2948) → `research.recommendation_log` (2951–2974)
Rumus/Logika	Return sejak tanggal rekomendasi per horizon; hit = return positif
Periode/Window	Horizon per log; cache 3600 s
Interaksi	—
Empty/Loading/Error	Skeleton / EmptyState bila log kosong

4. Data Flow (end-to-end)

```
EOD (prices_pit) + orderbook → analytics.recommendations (gate, skor, grade)
  → research.recommendation_log (dicatat per hari)
  → GET /api/recommendations | /track → SWR → Papan Kandidat + Track Record
```

5. Cara Pengambilan / Update Data

- **Perhitungan**: on-demand saat request; hasil mengikuti data EOD terbaru.
- **Pencatatan**: `recommendation_log` diisi saat kalkulasi (dasar Track Record).
- **Job**: tidak ada job khusus; ikut pipeline EOD (`refresh_s2` 16:05 WIB, `refresh_s2_late` 16:20 WIB).
- **Refresh UI**: rekomendasi 300 s; track record 3600 s.
- **Idempotensi**: log dedup per (tanggal, kode) [BELUM TERVERIFIKASI — cek constraint tabel `recommendation_log`].
- **Fallback**: data kosong → EmptyState (tanpa fallback sumber lain).

6. Dependensi & Relasi Menu

- Input: Screener (filter kandidat), Faktor & Kalibrasi (bobot).
- Output: dibaca Ruang Keputusan dan dicatat ke Jejak Sinyal.

7. Catatan Batasan & Edge Case

- Bukti pola smart money ditampilkan sebagai alasan, belum menimbang skor (data belum cukup panjang).
- Grade bersifat relatif terhadap pool hari itu; tidak bisa dibandingkan antar hari sebagai probabilitas.
- Alat bantu riset, bukan rekomendasi keuangan.

Ruang Keputusan (`/keputusan`, `/keputusan/[code]`)

1. Ringkasan Fungsi

Satu halaman untuk menjawab "apa posisi saya terhadap emiten ini". Menggabungkan skor keputusan dasar (lapisan verdict backend) sebagai dasar, lalu memperkaya dengan konteks yang **tidak mengubah verdict** (regime, sentimen aliran, jejak asing, base rate event, level pembatalan). Pengguna: trader/analis yang butuh ringkasan keputusan + alasan. Dipakai saat mau memutuskan hold/trim/exit satu emiten.

2. Route & Berkas

- URL: `/keputusan` (pemilih emiten) → `app/keputusan/page.tsx`; `/keputusan/[code]` → workspace
- Komponen: DecisionWorkspace.tsx (VerdictBadge, MiniBar, ContextStrip, ForeignMini, InvalidationLevels, StockPicker)
- Hook: `useStockDecision(code)` (hooks.ts, refresh 120 detik)

3. Penjelasan Per-Card

Verdict Dasar

Field	Isi
Nama card	Ruang Keputusan — `{code}`
Komponen	DecisionWorkspace.tsx (`VerdictBadge`, `MiniBar`)
Fungsi	Verdict gabungan teknikal + valuasi + lapisan IC/broker; skor N/100 + alasan
Sumber data	`GET /api/stocks/{code}/decision` (app.py:367) → `analytics.get_stock_decision` (analytics.py:1143); dasar dari `services.get_hold_check` (endpoint `/api/hold-check` tetap dipakai backend) (services.py:772)
Rumus/Logika	Verdict = skor dasar backend; `score = base_score + factor_adj + broker_adj`; verdict merek: STRONG HOLD / HOLD / TRIM / EXIT
Periode/Window	Verdict bergerak harian; cache 30 menit
Interaksi	Ganti emiten lewat StockPicker
Empty/Loading/Error	Empty "Emiten tidak ditemukan / histori tidak cukup untuk verdict"

Konteks Pasar & Aliran (Strip)

Field	Isi
Nama card	Context strip (4 tile)
Komponen	DecisionWorkspace.tsx (`ContextStrip`)
Fungsi	Regime IHSG (+ADX, as-of), Sentimen aliran (label + skor), Flip asing terakhir, Gauge pasar (N/100 + band)
Sumber data	`get_stock_decision` membaca `get_market_regime` (analytics.py:62) + `get_sentiment` (analytics.py:1984)
Rumus/Logika	Konteks murni informatif — tidak mengubah verdict
Periode/Window	Regime/sentimen terbaru
Interaksi	—
Empty/Loading/Error	"Tidak ada dasar" bila sentimen kosong

Jejak Asing 10 Sesi

Field	Isi
Nama card	Jejak Asing 10 Sesi
Komponen	DecisionWorkspace.tsx (`ForeignMini`)
Fungsi	Bar net asing per hari (rupiah), amber = hari flip
Sumber data	`get_stock_decision` → net asing per sesi
Rumus/Logika	Lebar bar ∝ `
Periode/Window	10 sesi terakhir
Interaksi	—
Empty/Loading/Error	Empty "Belum ada jejak asing."

Level Pembatalan

Field	Isi
Nama card	Level Pembatalan
Komponen	DecisionWorkspace.tsx (`InvalidationLevels`)
Fungsi	Level teknikal yang membatalkan tesis: Close, SMA50, Mean 60-hari, Bollinger bawah, RSI
Sumber data	`get_stock_decision` → `invalidation` (dari frame OHLC `prices_adjusted`)

Field	Isi
Rumus/Logika	Level dihitung dari harga; RSI ≥ 65 mendekati overbought, ≤ 35 oversold
Periode/Window	Snapshot terbaru
Interaksi	—
Empty/Loading/Error	Empty "Histori harga belum cukup untuk menghitung level."

Base Rate Event

Field	Isi
Nama card	Base Rate Event
Komponen	DecisionWorkspace.tsx
Fungsi	Median forward & abnormal return setelah event serupa; kolom Emiten (n×, tanggal), Median fwd, Median abn., Pasar (count · hit rate)
Sumber data	`get_stock_decision` → `events` (event study)
Rumus/Logika	Event study: riwayat emiten vs baseline pasar; sampel kecil = baca sekilas
Periode/Window	Per jenis event
Interaksi	—
Empty/Loading/Error	Card disembunyikan bila `events.length === 0`

4. Data Flow (end-to-end)

```
EOD ingest → Postgres (prices_pit / latest_pit, stock_summary_daily)
   → services.get_hold_check (skor) + analytics.get_sentiment + get_market_regime
   → analytics.get_stock_decision (gabung, cache 30m) → GET /api/stocks/{code}/decision
   → SWR useStockDecision (120s) → DecisionWorkspace render
```

5. Cara Pengambilan / Update Data

- **On-demand**: dihitung saat request, **bukan** precomputed job.
- **Cache**: 30 menit (`_research_cache`), kunci `stock_decision:{code}`.
- **Refresh UI**: SWR 120 detik.
- **Idempotensi**: verdict dari satu sumber (skor dasar backend); konteks dibaca ulang tiap request.
- **Fallback**: mengikuti skor dasar backend.

6. Dependensi & Relasi Menu

- Bergantung pada skor dasar backend (verdict), Sentimen, regime IHSG, dan event study.
- Menautkan ke `/stock/[code]` dan `/flow/[code]`.

7. Catatan Batasan & Edge Case

- Verdict butuh histori harga minimal; emiten tipis bisa kosong.
- Konteks (regime/sentimen/base rate) bersifat penilai kepercayaan — bukan pengubah verdict.
- Skor broker hanya muncul bila run IC tersedia.

Screener (`/screener`)

1. Ringkasan Fungsi

Mesin penyaring saham se-pasar berdasarkan kombinasi filter (momentum, RSI, volume, foreign, skor broker, order-book). Menjawab "saham mana yang cocok dengan kriteria saya hari ini". Pengguna: trader/analis yang mencari kandidat. Dipakai untuk riset kandidat sebelum masuk ke Rekomendasi/Ruang Keputusan.

2. Route & Berkas

- URL: `/screener` → `app/screener/page.tsx`
- Komponen: Card, RegimeBanner, TickerLogo, States
- Hooks: `useScreener`, `useBrokerActivity` (hooks.ts:216)

3. Penjelasan Per-Card

Preset & Panel Filter

Field	Isi
Nama card	Filter + preset (Momentum + Foreign In, Breakout Volume, Oversold Sehat, Trend Up Tenang, Akumulasi Broker)
Komponen	`PRESETS` /panel filter di screener/page.tsx:33
Fungsi	Menyusun parameter filter; preset mengisi beberapa filter sekaligus
Sumber data	Klien-only (state); dikirim sebagai query ke `GET /api/screener`
Rumus/Logika	Preset = kombinasi nilai filter (`min_momentum`, `foreign_in_only`, `min_vol_ratio`, `rsi_max`, `signal`, `min_broker_score`, `min_value`, `min_days`)
Periode/Window	`min_days` = syarat panjang data (default 30 sesi)
Interaksi	Klik preset / ubah filter → refetch
Empty/Loading/Error	—

Tabel Hasil Screener

Field	Isi
Nama card	Hasil (kolom: Close, RSI, Momentum 20d, Vol Ratio, ATR%, Jarak 52w, Hari sejak sinyal, Skor broker, OB imbalance, Absorption, Foreign net, Value)
Komponen	`COLUMNS` / `SortKey` di screener/page.tsx:84
Fungsi	Daftar emiten lolos filter + metrik pendukung; sortable & kolom bisa disembunyikan
Sumber data	`GET /api/screener` (app.py:527) → `analytics.get_screener` (analytics.py:1561) → `research.latest_pit`, `research.orderbook` (1744–1755)
Rumus/Logika	Momentum 20d = % perubahan 20 sesi; RSI(14); Vol ratio = volume÷rata-rata 20 sesi; ATR% = ATR(14)/close; Jarak 52w = posisi vs puncak 52 pekan; OB imbalance/absorption dari `research.orderbook`; Skor broker dari `research.factor_ic_history` (bobot lulus IC)
Periode/Window	20 sesi (momentum/volume), 14 (RSI/ATR), 52 pekan (jarak), lookback broker 60 hari
Interaksi	Klik header → sort; toggle kolom; klik emiten → detail
Empty/Loading/Error	Empty state bila tidak ada yang lolos; pesan khusus bila skor broker belum ada (butuh run `idx ic`)

Faktor (Legenda Skor)

Field	Isi
Nama card	Faktor (daftar definisi tiap kolom)
Komponen	`FACTORS` di screener/page.tsx:65
Fungsi	Menjelaskan arti & cara baca tiap kolom
Sumber data	Statis (teks)
Rumus/Logika	Definisi tiap faktor
Periode/Window	—
Interaksi	—
Empty/Loading/Error	—

4. Data Flow (end-to-end)

```
IDX EOD → ingest → Postgres (prices_pit, stock_summary_daily/foreign, orderbook capture)
  → analytics.get_screener (hitung indikator + join orderbook + skor broker)
  → GET /api/screener → SWR useScreener → tabel hasil + sort/filter
  (skor broker butuh research.factor_ic_history dari run `idx ic`)
```

5. Cara Pengambilan / Update Data

- **Perhitungan**: on-demand saat request (filter diterapkan di backend lalu di klien).
- **Job/scheduler**: tidak ada job khusus screener; bergantung pada job refresh EOD (`refresh_s2` 16:05 WIB, dll) yang mengisi `prices_pit` dan capture order-book saat jam bursa.
- **Refresh UI**: SWR (hook screener) 30 detik.
- **Idempotensi**: sumber data bitemporal (`prices_pit`) → hasil deterministik per as-of.
- **Fallback**: order-book kosong → kolom OB kosong; skor broker kosong sampai run IC ada.

6. Dependensi & Relasi Menu

- Bergantung pada job EOD (prices) + capture order-book saat jam bursa.
- Menyuplai kandidat ke Rekomendasi Beli dan Ruang Keputusan; skor broker berasal dari Faktor & Kalibrasi.

7. Catatan Batasan & Edge Case

- Filter `min_broker_score` menyaring semua bila kolom skor broker kosong (belum ada run `idx ic`).
- Butuh history minimal (`min_days`) sebelum indikator valid.
- Klasifikasi sektor/universe mengikuti mapping kurasi internal.

Sektor (`/sectors`)

1. Ringkasan Fungsi

Membandingkan performa antar sektor (perbankan, energi, konsumer, dan lain-lain) untuk melihat sektor mana yang sedang kuat dan ke mana rotasi dana bergerak. Pengguna: analis dan trader sektor. Sering lebih berguna daripada memilih satu saham.

2. Route & Berkas

- URL: `/sectors` → `app/sectors/page.tsx`
- Komponen: `RegimeBanner` (components/RegimeBanner.tsx), `SectorRotationTable` (components/SectorRotationTable.tsx), `Card`, `LastUpdated`
- Hook: `useSectors()` (hooks.ts:191, refresh 30 detik); `useSectorRotation(60, 3)` (hooks.ts:366, 300 detik) [BELUM TERVERIFIKASI baris hook]
- Backend: `GET /api/sectors` (app.py:507) → `get_sector_analysis` (analytics.py:1266); `GET /api/sectors/rotation?lookback=60&min_names=3` (app.py:493) → `get_sector_rotation` (analytics.py:2478); pemetaan sektor api/sector_map.py

3. Penjelasan Per-Card / Per-Panel

Regime IHSG (RegimeBanner)

Field	Isi
Nama card	Banner Regime
Komponen	RegimeBanner.tsx (baris 62 untuk volatilitas)
Fungsi	Konteks tren pasar sebelum membaca sektor
Sumber data	`GET /api/market/regime` (app.py:197) → `get_market_regime` (analytics.py:62)
Rumus/Logika	ADX(14) + DI pada IHSG; volatilitas = realized vol 20 hari, dianualisasi (~240 sesi)
Periode/Window	20 hari (volatilitas)
Interaksi	—
Empty/Loading/Error	Placeholder

Rotasi Sektor (SectorRotationTable)

Field	Isi
Nama card	Rotasi Sektor
Komponen	SectorRotationTable.tsx
Fungsi	Perubahan performa sektor dan pergeseran urutan (rotasi)
Sumber data	`GET /api/sectors/rotation` → `get_sector_rotation` (analytics.py:2478); cache `sector_rotation:{lookback}:{min_names}`
Rumus/Logika	Return sektor periode `lookback` (60 sesi); sektor dengan minimal 3 emiten (`min_names`)
Periode/Window	Lookback 60 sesi
Interaksi	— [BELUM TERVERIFIKASI interaksi klik]
Empty/Loading/Error	Pesan bila jumlah emiten per sektor kurang

Peta Sektor

Field	Isi
Nama card	Peta Sektor
Komponen	sectors/page.tsx baris 41 (subtitle "Diurutkan dari sektor terkuat")
Fungsi	Tabel per sektor: Rata-rata %, Breadth, Value, Foreign Net, Top Stock; warna sel (`heatColor`)
Sumber data	`GET /api/sectors` → `get_sector_analysis` (analytics.py:1266) via `useSectors`
Rumus/Logika	Rata-rata % perubahan harga emiten per sektor; breadth = porsi emiten naik; value = total nilai transaksi; foreign net = jumlah net asing sektor; top stock = kontributor terbesar
Periode/Window	Sesi terakhir (`date` opsional di backend)
Interaksi	Warna heat-map; sort menurut rata-rata
Empty/Loading/Error	`EmptyState`, `ErrorState` "Gagal memuat sektor", `Skeleton`

4. Data Flow (end-to-end)

```
Harga & volume EOD (stock_quotes/stock_daily, latest_pit) + foreign_net
  → sector_map (kode → sektor) → analytics.get_sector_analysis / get_sector_rotation
  → GET /api/sectors, /api/sectors/rotation → SWR (30 s / 300 s) → RegimeBanner + tabel + kartu
```

5. Cara Pengambilan / Update Data

- **On-demand** dengan cache in-process; data sumber dari pipeline EOD (`refresh_s2` 16:05, `refresh_s2_late` 16:20 WIB).
- **Refresh UI**: SWR 30 detik (peta) dan 300 detik (rotasi).
- **Reset cache**: `POST /api/cache/clear`.
- **Idempotensi**: deterministik dari data tersimpan.
- **Fallback**: sektor dengan emiten di bawah `min_names` tidak ditampilkan di rotasi.

6. Dependensi & Relasi Menu

- Konteks regime dari Dashboard.
- Foreign net per sektor berasal dari Foreign Flow.
- Emiten top sektor dapat dibuka ke Halaman Detail Emiten.

7. Catatan Batasan & Edge Case

- Pemetaan sektor bergantung pada `sector_map.py`; emiten tanpa pemetaan tidak masuk.
- Data EOD: sektor tidak berubah intraday.
- Sektor dengan emiten sedikit tidak stabil; gunakan `min_names`.
- [BELUM TERVERIFIKASI] Kolom persis `get_sector_analysis` dan logika breadth; cek analytics.py:1266.

Sentimen (`/sentimen`)

1. Ringkasan Fungsi

Membaca suasana pasar dari data yang sudah ada: arus asing, ketimpangan buku order intraday, dan breadth. Menghasilkan gauge pasar dan daftar emiten akumulasi/distribusi. Pengguna: trader yang ingin tahu apakah pasar sedang optimis atau pesimis. Tidak ada sumber berita atau scraping baru.

2. Route & Berkas

- URL: `/sentimen` → `app/sentimen/page.tsx`
- Hook: `useSentiment(limit=15)` (hooks.ts:204, refresh 300 detik)
- Backend: `GET /api/sentiment?limit=15` (app.py:574) → `get_sentiment` (analytics.py:1984) → `research/sentiment.compute` (research/sentiment.py)

3. Penjelasan Per-Card / Per-Panel

Gauge Pasar

Field	Isi
Nama card	Gauge Pasar
Komponen	sentimen/page.tsx ~baris 154–190
Fungsi	Skor suasana pasar 0–100 (jarum ke kanan = optimis)
Sumber data	`GET /api/sentiment` → `sentiment.compute(dsn, index_pct, ob_by_code, limit)` (research/sentiment.py); `index_pct` dari `_latest_index` (analytics.py); `ob_by_code` dari `research.orderbook`
Rumus/Logika	Gabungan arus asing (`latest_pit.foreign_net`), ketimpangan & absorpsi buku order, dan breadth; dinormalisasi ke 0–100 [BELUM TERVERIFIKASI bobot komponen]
Periode/Window	Snapshot hari bursa terakhir (`as_of`)
Interaksi	Hover tooltip (info)
Empty/Loading/Error	Skeleton; ErrorState "Pastikan backend & database hidup"

Akumulasi

Field	Isi
Nama card	Akumulasi
Komponen	sentimen/page.tsx ~baris 191–201; `SentimentList`
Fungsi	Emiten dengan arus asing peringkat atas dan/atau bid lebih tebal dari offer
Sumber data	Field `accumulation` pada respons `/api/sentiment`
Rumus/Logika	Skor emiten −1..+1 dari komponen sentimen; ambil `limit` teratas
Periode/Window	Snapshot hari terakhir
Interaksi	Klik emiten → detail
Empty/Loading/Error	"Tidak ada emiten dengan sinyal akumulasi."

Distribusi

Field	Isi
Nama card	Distribusi
Komponen	sentimen/page.tsx ~baris 202–212
Fungsi	Emiten dengan arus asing peringkat bawah dan/atau offer lebih tebal dari bid
Sumber data	Field `distribution`
Rumus/Logika	Skor terendah dari komponen yang sama
Periode/Window	Snapshot hari terakhir
Interaksi	Klik emiten → detail
Empty/Loading/Error	"Tidak ada emiten dengan sinyal distribusi."

4. Data Flow (end-to-end)

```
latest_pit.foreign_net + research.orderbook (intraday, tertutup close T) + index_quotes
  → research/sentiment.compute → cache 1 jam (analytics)
  → GET /api/sentiment?limit=15 → useSentiment (300 s) → gauge + dua daftar
```

5. Cara Pengambilan / Update Data

- **On-demand** dengan cache `sentiment:{limit}` 1 jam (`_RESEARCH_TTL`, analytics.py:37).
- **Sumber orderbook**: diisi capture intraday (lihat README bagian scheduler: `intraday`, interval 60 detik) [BELUM TERVERIFIKASI job persis].
- **Refresh UI**: SWR 300 detik; data tetap sampai cache 1 jam habis.
- **Reset cache**: `POST /api/cache/clear`.
- **Idempotensi**: deterministik dari data tersimpan.
- **Fallback**: bila `orderbook` gagal, `ob_by_code` kosong dan sentimen hanya dari arus asing dan breadth (warning di log).

6. Dependensi & Relasi Menu

- Memakai data Foreign Flow dan indeks dari Dashboard.
- Daftar emiten dapat dibuka ke Halaman Detail Emiten.

7. Catatan Batasan & Edge Case

- Snapshot intraday hari T ditutup pada close T; tidak ada look-ahead.
- Sentimen bukan berita: tidak ada headline atau media sosial.
- Tanpa capture order-book, gauge tetap tampil dengan komponen lebih sedikit.
- Cache 1 jam: angka bisa tertinggal hingga 1 jam.

Sumber Data Berita Market

Dokumen ini adalah daftar sumber berita dan data untuk Analis Sentimen dan Analis Berita. Peran agen ada di framework-agen.md.

Prioritas: pasar Indonesia lebih dulu, lalu makro global, lalu crypto dan emas sebagai pelengkap.

1. Pasar Indonesia (IHSG, saham, obligasi, makro BI)

Sumber	Fokus	Catatan
idx.co.id	Data resmi BEI: pengumuman, laporan keuangan	Sumber primer. Diakses lewat browser (Cloudflare)
cnbcindonesia.com	Berita market dan analisis institusional	
bisnis.com	Berita pasar modal domestik	
kontan.co.id	Berita pasar modal domestik	
investor.id	Berita pasar modal domestik	
investing.com	Section Indonesia	
Pluang	Data dan berita ritel	

2. Global Macro & Institutional

Sumber	Fokus
reuters.com/markets	Berita pasar global
bloomberg.com	Berita pasar global
marketwatch.com	Berita pasar AS
tradingeconomics.com	Data makro: BI rate, Fed rate, inflasi

3. Crypto (BTC/ETH/altcoin + sentimen)

Sumber	Fokus
tradingview.com	Chart dan data harga
coindesk.com, cointelegraph.com	Berita dan analisis
theblock.co	Berita kelas institusional
coinglass.com	On-chain, funding rate, whale dan likuidasi (data, bukan berita)
alternative.me	Fear & Greed Index harian

4. Gold / Komoditas

Sumber	Fokus
kitco.com	Berita dan harga emas real-time. Sering di-scrape
goldprice.org	Harga historis dan live
Pegadaian Gold, UBS Gold	Harga emas domestik dan institusi

5. Sentimen Sosial (untuk "whale mode" sentiment read)

Sumber	Fokus
Threads, X, Stockbit	Saham
StockTwits, Reddit	Saham dan crypto global

Catatan

- Sebelum menambah sumber baru, cek dulu terms of service dan apakah sumber tersebut memang dipakai di kode (`idx-scraper`).
- Status scraping per sumber belum diverifikasi di dokumen ini. Tandai `[BELUM TERVERIFIKASI]` sampai ada bukti.